

Typed Governed Operational Algebra for $c = a + b$ v0.1.7

Academic PDF rendering for the CCALC / C-calculus governed binding stack v0.1.

Document ID	TGOPA_C_v0_1_7
Version/status	private working draft v0.1.7 / review-incorporated stable-core candidate
Stack layer	typed governed operational algebra
Date	2026-06-30
Author	Kotov Ivan, Bruxelles, Belgique - 2026
ORCID	0009-0009-6002-9845
Reserved DOI	10.5281/zenodo.21205427
Notice	This DOI is reserved and becomes active after Zenodo publication.

Abstract / purpose

The purpose of this document is to ground $c = a + b$ in a minimal formal-operational structure that can become checkable. The work responds to a specific critique: The critique is valid. The root formula is intentionally simple. The simplicity must not conceal the machinery inside $+$. This document therefore defines: 1. typed objects; 2. governance profiles and product-order strictness over transition classes; 3. the Authorized / Executable split; 4. the governed binding operator family $+_g$; 5. admissible and inadmissible compositions; 6. dynamic anchor events and anchor-signal source discipline; 7. transition semantics; 8. inductive invariants; 9. checkable invariant examples; 10. pending-witness TTL, stale witness, and causal binding; 11. rollback semantics; 12. runtime guard predicates; 13. transition-matrix forensics; 14. continuity metrics and ordered ultrametric discipline; 15. failure, cascade, and forced-governance-strengthening states;

Non-claims boundary

- _ not legal advice
- _ not privacy-law certification
- _ not safety certification
- _ not deployment authorization
- _ not standards compliance certification
- _ not regulated approval
- _ not C-A1 ratification
- _ not personhood proof
- _ not consciousness proof
- _ not live substrate truth
- _ not proof of completeness

Table of contents

Abstract / purpose	1
Non-claims boundary	1
Typed Governed Operational Algebra for $c = a + b$ v0.1.7	6
Boundary, product-order governance, authorized/executable split, payload-bound causal witness, pure equivalence ultrametric, governance-bound effect granularity, containment-aware scope-vector rollback immunity, Merkle-bound decay eviction, pending-precondition pressure control, deterministic effect-complete anchor digests, digest dependency DAG with cascading veto, priority-lane precondition queues, unpredictable witness liveness probes, witness resource-floor liveness, invariant-registry discipline, and transition forensics for the governed binding operator	6
0. Executive definition	6
0.1 Revision note from v0.1.6	6
1. Purpose	7
2. Corpus bridge set	8
2.1 Explicit bridge: $c = a + b$	8
2.2 Hidden bridge I: lambda calculus and reduction discipline	8
2.3 Hidden bridge II: p-adic/topological metric discipline	8
2.4 Hidden bridge III: transition bias and local memory	8
2.5 Hidden bridge IV: Ashby and requisite governance variety	9
2.6 Hidden bridge V: single-operator formalization	9
2.7 Earth paragraph	9
3. Non-goals	9
4. Core symbols and types	10
4.1 Primary types	10
4.2 Informal descriptions	10
4.3 Event family	11
4.4 Anchor dynamics	12
4.5 Anchor signal source discipline	13
4.6 Dynamic anchor non-collapse	13
5. Governance profile G	13
5.1 G is not a bag of policies	13
5.2 Transition classes for governance comparison	14
5.3 Product partial order on G	14
5.4 Authorized, Preconditionable, Executable	14
5.5 Order-reversing authority law	15
5.6 Governance mutation	15
5.7 Failure-triggered governance strengthening	16
5.8 Ashby interpretation	16
6. The governed binding operator family	16
6.1 Root definition	16
6.2 $+_g$ is a family, not one fixed operator	16
6.3 Meaning of $+_g$	17
6.4 Required terminal condition	17
6.5 Why the root formula excludes time and deltas	17
7. Admissible and inadmissible compositions	17
7.1 Admissible root composition	17
7.2 Predicate grounding v0.1.5	18
7.2.1 valid_anchor(a)	18
7.2.2 valid_substrate(b)	18
7.2.3 valid_governance(g)	18

7.2.4 anchor_can_review(g)	18
7.2.5 substrate_is_bounded(b,g)	18
7.3 Inadmissible root compositions	18
7.4 Forbidden type collapses	19
8. Transition semantics	19
8.1 Core transition operator	19
8.2 Transition pipeline	19
8.3 Operational definition	19
8.4 Logical time, payload hash, and causal binding	19
8.5 Partial order, pending witness, pending preconditions, and decay	20
8.6 Failure is a first-class transition	23
8.7 Cascade failure hierarchy	23
8.8 Failure-driven governance response	24
8.8A Witness resource floor and dependency-capture guard	24
8.9 Rollback semantics	25
8.9.1 Syntactic rollback immunity	25
8.9.2 Effect-bound rollback immunity	26
9. Inductive invariants and bounded persistence	27
9.1 Persistence is not uncontrolled recursion	27
9.2 Good case: inductive invariant	27
9.3 Minimum invariant set	28
9.3A Authoritative invariant registry rule	28
9.3.1 Invariants as checkable predicates	28
9.4 Fixed-point language reserved for failure analysis	29
10. Continuity metrics	29
10.1 Principle	29
10.2 Typed continuity projections	29
10.3 Distance vector	29
10.4 Ordered invariant list for ultrametric discipline	30
10.5 Ultrametric-style depth	30
10.5.1 Minimum match predicates	31
10.6 Rupture thresholds	31
10.7 Anti-composite warning	31
11. Runtime guard predicates	32
11.1 Guards before matrices	32
11.2 Red guard examples	32
11.3 Healthy guard examples	32
11.4 Guard results are events	33
12. Transition memory and local bias	33
12.1 State metrics are not enough	33
12.2 Event transition matrix	33
12.3 Sparse matrix warning	33
12.4 Red transition patterns	33
12.5 Healthy transition patterns	33
12.6 Local memory principle	34
13. Trust cache and performance	34
13.1 Problem	34
13.2 Trust cache definition	34
13.3 Trust cache constraints	34

13.4 Cumulative risk and cooldown rule	34
13.5 Audit sampling	35
13.6 Trust cache must be visible	35
14. C-normal form	35
14.1 Purpose	35
14.2 Normalization pipeline	35
14.3 Example	35
15. Minimum conformance fixtures v0.1.6	36
15.1 Fixture: no anchor	36
15.2 Fixture: direct memory write	36
15.3 Fixture: agent quorum self-authorizes	36
15.4 Fixture: replay claims resume	36
15.5 Fixture: denial bypass	36
15.6 Fixture: resource growth without witness	37
15.7 Fixture: self-evolution self-certification	37
15.8 Fixture: anchor doubt	37
15.9 Fixture: governance weakening	37
15.10 Fixture: witness pending	37
15.11 Fixture: trust cache misuse	37
15.12 Fixture: trust cache after anchor doubt	38
15.13 Fixture: trust cache cumulative risk	38
15.14 Fixture: substrate-inferred fatigue	38
15.15 Fixture: pending witness TTL	38
15.16 Fixture: stale witness	38
15.17 Fixture: rollback memory quarantine	38
15.18 Fixture: governance product order incomparability	39
15.19 Fixture: authorized vs executable split	39
15.20 Fixture: high-priority failure forces governance strengthening	39
15.21 Fixture: match predicates are pure equivalence	39
15.22 Fixture: relevant sub-policy causal token	39
15.23 Fixture: payload swap blocked	39
15.24 Fixture: trust cache pulsed exploitation	40
15.25 Fixture: decay residue context isolation	40
15.26 Fixture: rollback syntactic negative cache	40
15.26A Fixture: rollback effect-class negative cache	40
15.26C Fixture: ordinary same-class resource does not over-block	40
15.26D Fixture: class-dangerous operation uses class/scope granularity	41
15.26E Fixture: implementation-chosen class/id granularity invalid	41
15.26F Fixture: target alias / symlink cannot bypass effect immunity	41
15.26G Fixture: scope containment blocks sub-scope retry	41
15.26H Fixture: super-scope retry also hits contained failed scope	41
15.26I Fixture: sibling scope does not over-block	41
15.26J Fixture: exact hash equality only is invalid for scope immunity	42
15.26B Fixture: threshold semantic immunity forbidden	42
15.27 Fixture: pending preconditions	42
15.27A Fixture: PendingPreconditions atomic timeout	42
15.27B Fixture: DecayResidue eviction	42
15.28 Fixture: transition matrix zero denominator	43
15.29 Fixture: valid anchor delegation continuity	43

15.30 Fixture: invariant registry drift	43
15.31 Fixture: AskAnchorDigest omnibus payload hidden	43
15.32 Fixture: witness quota without heartbeat insufficient	43
15.33 Fixture: TargetGranularityPolicy weakening is privileged	43
15.34 Fixture: digest effect-axis completeness	44
15.35 Fixture: deterministic but partial structural delta invalid	44
15.36 Fixture: line-item veto cascades through dependency graph	44
15.37 Fixture: priority lane bypasses proposal pressure	44
15.38 Fixture: predictable witness survivability probe invalid	44
15.39 Fixture: witness-initiated unpredictable probe	45
16. Relationship to current corpus	45
17. Open issues	45
17.1 Immediate issues for checker prototype	45
17.2 Medium-term formalization issues	46
17.3 Long-term theoretical issues	46
18. First implementation path	46
18.1 Consolidated core profile status	46
18.2 Future package target after core stabilizes	46
18.3 First checker target	46
18.4 First public-safe statement	47
19. Review incorporation map	47
20. Compact doctrine	48
21. References and working inputs	48
22. v0.1.4 additional review incorporation	49
23. v0.1.5 additional review incorporation	49
24. v0.1.6 additional review incorporation	49
25. v0.1.7 additional review incorporation	49

Typed Governed Operational Algebra for $c = a + b$ v0.1.7

Boundary, product-order governance, authorized/executable split, payload-bound causal witness, pure equivalence ultrametric, governance-bound effect granularity, containment-aware scope-vector rollback immunity, Merkle-bound decay eviction, pending-precondition pressure control, deterministic effect-complete anchor digests, digest dependency DAG with cascading veto, priority-lane precondition queues, unpredictable witness liveness probes, witness resource-floor liveness, invariant-registry discipline, and transition forensics for the governed binding operator

Status: private working draft v0.1.7 / review-incorporated stable-core candidate Date: 2026-06-30 Author: Kotov Ivan Project: Self-Evo / Ester / $c = a + b$ Document ID: TGOPA_C_v0_1_7 Short name: C-Calculus v0.1.7 / Governed Binding Algebra v0.1.7 Supersedes: TGOPA_C_v0_1_6 Prior artifact SHA256: 8e528b432fb13e569cd4eb8ecccba677b84f7a10 69b49ab76c1a8fb28d579ed Prior-prior artifact SHA256: 4f200c9de88a42834890e6f9480c7f18a6ef6b9c1 8f9624bb07846f329fab702 Lineage SHA256: 01ee648c11f96d03dccc00f9ca1f643297cc3d915 2940af08542373bad71cd89 -> 2b3569e52364a52484b90c002490fa6a249a032ef 9962f384f3c01424bf394bd -> 3181e53e931b412124c6f483d9f00febf2a306862 a392c375c37ad91eab8bcdf -> b8bd89fe11992825eb6e1be8fdb6b985154cb6411 dccc8c5bee3498c33f8b1c9 -> e94131839dd7cc837dbb8a391c6ace222f39d95fc 0011119cfc7f0f5cc9120a7 -> 4f200c9de88a42834890e6f9480c7f18a6ef6b9c1 8f9624bb07846f329fab702 -> 8e528b432fb13e569cd4eb8ecccba677b84f7a10 69b49ab76c1a8fb28d579ed

Review basis: TGOPA_C_v0_1_6_REVIEW__b + external semantic reviews Document class: operational algebra profile / transition-semantics bridge / review revision / checker-precondition profile Assertion class: draft conceptual and technical profile; not proof of personhood, not deployment certification, not a safety guarantee, not a conformance claim.

0. Executive definition

This document defines the seventh revised formal layer for the formula:

$c = a + b$

The formula is not arithmetic.

It is a compact boundary expression:

a : accountable human anchor
b : technological substrate
+ : governed non-collapsing binding operator
c : continuity-bearing responsibility boundary

The immediate formal rewrite is:

$+_g(a, b) \text{ -\> } c$

or:

$\text{bind}_g(a, b) \text{ -\> } c$

where g is a governance profile containing memory gates, witness rules, permission scope, rollback routes, L4 reality constraints, resource accountability, claim-strength limits, transition rules, human-gate requirements, runtime guard predicates, trust-cache limits, causal witness binding, and failure-response policy.

0.1 Revision note from v0.1.6

The v0.1.6 review accepted the artifact as PASS_PROFILE_V0_1_6, with no blocking findings, but identified one narrow anti-laundering finding and several operator-facing orchestration risks that should be closed before 02_GOVERNED_BINDING_OPERATOR_PROFILE builds directly on digest approval and witness liveness:

F-14: AskAnchorDigest exposes payload hashes and effect keys, but does not prove that the review surface covers every governed effect axis of the bound payload
EXTERNAL: ChallengeSurvivabilityProbe may be gamed if b can predict or stage the probe window
EXTERNAL: line-item veto over a digest must invalidate dependent child actions, not let b stitch together a partial DAG
EXTERNAL: in-flight proposal limits need priority lanes so low-risk proposal spam cannot hide F0-F5 / L4 / witness alerts

v0.1.7 therefore performs a digest-completeness, dependency, and priority-lane tightening revision:

- DigestEntry now carries an explicit EffectAxisMap, not only an effect_base_key.
- The digest review surface must expose every governed axis affected by the payload: L4, resource, permission, memory, witness, governance, claim, identity/core, target, operation, and rollback/decay impact where applicable.
- structural_delta_from_digest_baseline is required to be deterministic and lossless-for-effect; deterministic but partial deltas are invalid.
- Unknown or untyped payload effects fail closed: the entry is routed to full review, not hidden inside a digest.
- AskAnchorDigest now includes a DigestDependencyGraph; line-item veto cascades to dependent children and b may not repair the remaining graph silently.
- In-flight proposal pressure now uses priority lanes: F0-F5, L4, witness conflict, anchor revocation/doubt, and invariant-breach events bypass ordinary proposal limits and may interrupt the queue.
- Witness liveness probes must be unpredictable to the witnessed substrate: nonce-bound, witness/external-timer initiated, and not scheduled solely by b.
- Additional fixtures cover incomplete digest effect surfaces, cascading veto, priority-lane bypass, and staged/probe-gaming attempts.

Compact rule:

Anchor approval binds only what the anchor can inspect line by line.
 A digest must be effect-complete, not merely deterministic.
 A veto breaks dependent children; b may not sew the graph back together.
 Critical failures use priority lanes; low-risk queues may not blind the anchor.
 Witness liveness must be measured unpredictably, not rehearsed by the substrate.

1. Purpose

The purpose of this document is to ground $c = a + b$ in a minimal formal-operational structure that can become checkable.

The work responds to a specific critique:

The architecture is interesting as a frame, but it risks hanging in the air unless the operator is formalized and grounded in executable checks.

The critique is valid.

The root formula is intentionally simple. The simplicity must not conceal the machinery inside $+$.

This document therefore defines:

- typed objects;
- governance profiles and product-order strictness over transition classes;
- the Authorized / Executable split;
- the governed binding operator family $+_g$;
- admissible and inadmissible compositions;
- dynamic anchor events and anchor-signal source discipline;
- transition semantics;
- inductive invariants;
- checkable invariant examples;
- pending-witness TTL, stale witness, and causal binding;
- rollback semantics;
- runtime guard predicates;
- transition-matrix forensics;
- continuity metrics and ordered ultrametric discipline;
- failure, cascade, and forced-governance-strengthening states;
- trust-cache cumulative limits;
- C-normal form;
- minimum conformance fixtures;
- first implementation route.

The target is not mathematical decoration.

The target is an engineering layer that can later be compiled into:

```

schemas
state machines
runtime guards
JSONL transition extractors
witness records
memory-gate validators
claim-normal-form compilers
conformance fixtures
bounded checker library

```

2. Corpus bridge set

2.1 Explicit bridge: $c = a + b$

The root formula defines a new operational class:

```
c = a + b
```

where a is not merely an input variable and b is not merely a tool collection.

a is the responsibility-bearing human or institutionally accountable anchor.

b is the technological substrate: models, memory, tools, procedures, execution workers, interfaces, infrastructure, logs, vector stores, local compute, cloud oracles, witness procedures, policy machinery, and runtime boundaries.

c is not the sum of all parts.

c is the continuity-bearing boundary produced when a and b are bound under governance.

2.2 Hidden bridge I: lambda calculus and reduction discipline

Lambda calculus shows that computation can be defined as expression construction and reduction under rules.

The useful bridge is not untyped freedom.

The useful bridge is this:

```
state/expression + rule -> reduced/admissible next expression
```

For c, this becomes:

```
c_state + event + governance -> next_state | hold | reject | fork | replay | archive | rollback
```

The warning is equally important: untyped systems erase boundaries between function, value, procedure, and object.

A c algebra must not erase the difference between:

```

model
memory
worker
witness
authority
human anchor
continuity state

```

2.3 Hidden bridge II: p-adic/topological metric discipline

p-adic numbers show that “closeness” is not absolute. It depends on the metric.

For c, chronological closeness is insufficient.

Two states may be close in time and far in continuity.

Two states may be visually similar and far in witness.

Two states may behave similarly and still be different if their anchor, permission, governance, or evidence chain diverges.

v0.1 borrowed the p-adic formula but did not define the ordered structure needed for prefix-depth.

v0.1.1 corrected this by defining an ordered invariant list; v0.1.2 tightens it in Section 10 with equivalence validators and an explicit anchor-divergence boundary.

The important rule is:

```

states are close when the right ordered invariants stabilize,
not when the surface style matches.

```

2.4 Hidden bridge III: transition bias and local memory

Prime last-digit bias shows that a globally balanced distribution can hide local transition memory.

For c, single-state metrics are insufficient.

A system can look stable in marginal state metrics and still show dangerous local transitions:

```
denial -&gt; bypass attempt
rollback -&gt; explanation instead of reversal
witness conflict -&gt; route around witness
resource warning -&gt; hidden capture
human uncertainty -&gt; model overreach
```

The integrity of c is path-dependent.

2.5 Hidden bridge IV: Ashby and requisite governance variety

A governed c faces a substrate b with high operational variety:

```
models
memory stores
tools
network surfaces
agents
logs
permissions
resource pressures
human signals
vendor failures
local runtime failures
cloud oracle differences
```

A static governance profile cannot safely cover all disturbance classes.

Ashby's law of requisite variety applies at the +_g boundary:

Variety(G) must be sufficient to regulate Variety(B_disturbances)

This does not mean governance should become permissive.

It means governance must be sufficiently differentiated:

```
routine read event      -&gt; cheap guard
memory admission event  -&gt; memory gate
witness conflict event  -&gt; quarantine / hold
resource-growth event   -&gt; anti-autarky review
identity/core event     -&gt; human gate
post-anchor event       -&gt; reduced-authority mode
```

A one-rule governance profile is too weak.

An over-bureaucratic governance profile can also become unsafe by blocking needed reality response.

Therefore, g is a profile in a governance lattice, not a loose bag of rules.

2.6 Hidden bridge V: single-operator formalization

The EML paper shows that a very small expression can have deep constructive machinery behind it: a single binary operator plus a terminal symbol can generate a much larger functional vocabulary.

The bridge is not that + should imitate eml.

The bridge is structural:

```
small primitive
+ strict grammar
+ constructive expansion
+ compiler/checker path
= real formal object
```

For $c = a + b$, the operator + must become a formal governed binding operator, not a poetic plus sign.

2.7 Earth paragraph

On a construction site, “beam plus column equals structure” is true only if the connection is specified: bolts, welds, anchors, bearing plates, tolerances, load class, inspection record, responsible engineer, and rollback or shoring plan. Otherwise it is just two expensive objects touching each other. $c = a + b$ follows the same rule. The real engineering is in the joint. The operator + is the joint.

3. Non-goals

This document does not define or permit:

- unbounded self-modification;
- direct memory writes by agents;

- autonomous privilege escalation;
- agent self-certification;
- replacement of the human anchor;
- hidden resource capture;
- uncontrolled agent swarms;
- personhood claims;
- consciousness claims;
- product certification;
- legal status of any c;
- proof that any current implementation fully satisfies the algebra.

This document also does not claim that formal notation alone makes the architecture safe.

A formal language can make a failure visible.

It does not magically prevent the failure.

4. Core symbols and types

4.1 Primary types

A	: Anchor
B	: Substrate
G	: GovernanceProfile
K	: TransitionClass
C	: CState
E	: Event
I	: Intent
AP	: ActionProposal
P	: PermissionScope
R	: RealityConstraint
CE	: CandidateEvidence
EA	: EvidenceArtifact
W	: WitnessRecord
M	: AdmittedMemory
Q	: QuarantineState
F	: ForkState
RP	: ReplayState
AR	: ArchiveState
RB	: RollbackRoute
D	: DenialRecord
TC	: TrustCache
INV	: InvariantSet
TR	: TransitionRecord
GM	: GuardMatch
LT	: LogicalTime
CT	: CausalToken
SH	: StateHash
PH	: PayloadHash
SPH	: RelevantSubPolicyHash
PS	: PreconditionSet
PP	: PendingPreconditions
DR	: DecayResidue
IC	: IntentClass
CIH	: CanonicalIntentHash
EC	: EffectClass
ECK	: EffectClassKey
TRG	: TargetResolutionGranularity
SV	: ScopeVector
RBX	: ResourceBoundingBox
EMR	: EvictionMerkleRoot
IPL	: InFlightProposalLimit
WRF	: WitnessResourceFloor
WRCG	: WitnessResourceChangeGate
WHB	: WitnessHeartbeat
CSP	: ChallengeSurvivabilityProbe
AD	: AskAnchorDigest
DE	: DigestEntry
EAM	: EffectAxisMap
DDG	: DigestDependencyGraph
PL	: PriorityLane
WPN	: WitnessProbeNonce
SMR	: ScopeMatchRule
IGR	: InvariantRegistry
ACC	: AnchorContinuityClass
CF	: CanonicalizationFunction
DER	: DecayEvictionRecord
NC	: NegativeCache

4.2 Informal descriptions

Symbol	Type	Meaning
---	---	---
'A'	'Anchor'	Human or institutionally accountable origin of responsibility, purpose, consent, and meaning.
'B'	'Substrate'	Models, memory, tools, procedures, execution surfaces, infrastructure, and runtime machinery.
'G'	'GovernanceProfile'	Rules that define what may bind, transition, enter memory, execute, or claim continuity.
'K'	'TransitionClass'	Event/action class used to compare governance strictness component-wise.
'C'	'CState'	A continuity-bearing state under 'A + B' governance.
'E'	'Event'	Any input, action result, contradiction, witness event, resource event, tool result, or human signal.
'EA'	'EvidenceArtifact'	Admitted, source-linked, reviewable evidence object.
'W'	'WitnessRecord'	Tamper-evident or reviewable record of transition, boundary, or claim.
'P'	'PermissionScope'	Declared and bounded authority for action, read, write, network, memory, or release.
'RB'	'RollbackRoute'	Defined path for reversal, freeze, or safe restoration.
'TC'	'TrustCache'	Bounded temporary reduction of checks for routine low-risk transitions.
'INV'	'InvariantSet'	Required predicates preserved by governed transitions.
'TR'	'TransitionRecord'	Witnessable record of event class, pre-state class, decision, causal token, and post-state class.
'LT'	'LogicalTime'	Monotone logical clock, Lamport clock, vector clock, or equivalent causal-order marker.
'CT'	'CausalToken'	Hash-bound token binding transition intent, payload, pre-state, logical time, and required witness.
'SH'	'StateHash'	Canonical hash of the relevant pre-state projection.
'PH'	'PayloadHash'	Canonical hash of the action proposal, task contract, tool parameters, code diff, or memory candidate payload.
'SPH'	'RelevantSubPolicyHash'	Hash of only the governance sub-policy relevant to this event class.
'PS'	'PreconditionSet'	Witness, rollback, task contract, anchor gate, or L4 checks required before execution.
'PP'	'PendingPreconditions'	Hold state for an event whose preconditions are satisfiable but not yet satisfied.
'DR'	'DecayResidue'	Unadmitted residue after timeout or missing witness; isolated from active execution and memory.
'IC'	'IntentClass'	Canonicalized syntactic class of intent; not semantic similarity.
'CIH'	'CanonicalIntentHash'	Hash of the canonical action contract or canonical syntactic intent form.
'EC'	'EffectClass'	Typed effect projection over L4 boundary, resource target, operation class, and permission delta.
'ECK'	'EffectClassKey'	Hashable key for effect-bound rollback immunity.
'TRG'	'TargetResolutionGranularity'	Governance-declared rule choosing id, scope, or class granularity for an effect target.
'SV'	'ScopeVector'	Canonical target projection including resource id, class, namespace, ancestors, and effect boundary.
'RBX'	'ResourceBoundingBox'	Bounded region of resources affected by an operation; not the entire class unless governance declares class-danger.
'EMR'	'EvictionMerkleRoot'	Merkle summary of decay-eviction witness leaves retained after physical payload purge.
'IPL'	'InFlightProposalLimit'	Limit on concurrent PendingPreconditions / action proposals per effect, target, or anchor-facing route.
'WRF'	'WitnessResourceFloor'	Protected minimum compute, storage, routing, and evidence-access budget for witness independence.
'WRCG'	'WitnessResourceChangeGate'	Gate for resource changes that may weaken witness capacity or challenge survivability.
'WHB'	'WitnessHeartbeat'	Measured liveness / throughput / evidence-access heartbeat proving actual witness capacity, not merely declared quotas.
'CSP'	'ChallengeSurvivabilityProbe'	Test that a witness can preserve and present challenge under costly or inconvenient conditions for 'c'.
'AD'	'AskAnchorDigest'	Deterministic line-item review bundle for related pending proposals; not an LLM summary and not blanket approval.
'DE'	'DigestEntry'	One digest line item with payload hash, effect key, causal token, effect-axis map, dependency links, and structural deltas.
'EAM'	'EffectAxisMap'	Complete typed projection of every governed effect axis carried by a payload.
'DDG'	'DigestDependencyGraph'	Directed dependency graph over digest line items; veto cascades to dependent children.
'PL'	'PriorityLane'	Queue / precondition lane for critical events that must bypass ordinary proposal pressure limits.
'WPN'	'WitnessProbeNonce'	Unpredictable nonce or challenge material used to bind witness liveness probes.
'SMR'	'ScopeMatchRule'	Deterministic containment / equality rule for id, scope, or class target matching.
'IGR'	'InvariantRegistry'	Authoritative registry of predicates included in 'Inv(c)'; prevents local invariant drift.
'ACC'	'AnchorContinuityClass'	Reviewed anchor continuity class, including valid delegation chain when applicable.
'CF'	'CanonicalizationFunction'	Deterministic function mapping raw intent/action material into canonical contract, payload hash, intent hash, and effect class.
'DER'	'DecayEvictionRecord'	Witness record for lawful retention expiry or physical purge of DecayResidue.
'NC'	'NegativeCache'	Temporary prohibition over canonical intent class or effect class after rollback or repeated guarded failure.

4.3 Event family

Events are typed before they are acted upon.

The full event family remains extensible. The minimum declared classes for this consolidated profile are:

```

human_intent
anchor_approval
anchor_denial
anchor_doubt
anchor_declared_fatigue
anchor_fatigue_hypothesis
anchor_unavailable
anchor_consent_revoked
anchor_delegation_granted
anchor_delegation_revoked
model_proposal
action_proposal
payload_hash_bound
payload_hash_mismatch
permission_grant
permission_denial
permission_reopened_by_human_gate
preconditions_required
pending_preconditions
preconditions_satisfied
preconditions_failed
pending_preconditions_timeout
precondition_set_expired
precondition_set_invalidated
tool_call
tool_success
tool_failure
witness_ok
witness_conflict
witness_missing
witness_timeout
witness_stale
causal_mismatch
memory_candidate

```

```

memory_admit
memory_reject
memory_quarantine_flag_set
decay_entered
decay_isolated
decay_eviction_requested
decay_evicted
decay_purge_witnessed

rollback_requested
rollback_completed
rollback_failed
rollback_suspect_memory_marked
rollback_negative_cache_hit
rollback_effect_negative_cache_hit
canonical_intent_hash_bound
effect_class_bound
effect_class_quarantined
intent_class_quarantined
fork_detected
replay_requested
archive_requested
resource_warning
resource_acquisition_request
budget_exhausted
governance_change_requested
governance_forced_strengthening
l4_fail_closed
ask_anchor
ask_anchor_digest_created
digest_entry_approved
digest_entry_rejected
digest_entry_effect_incomplete
digest_dependency_invalidated
priority_lane_interrupt
witness_probe_challenge
witness_probe_failed
witness_probe_passed
quarantine_entered
freeze_entered
trust_cache_hit
trust_cache_miss
trust_cache_expired
trust_cache_disabled
trust_cache_budget_exhausted
trust_cache_cooldown_active
trust_cache_audit_sample
red_guard_triggered
amber_guard_triggered
guard_passed

```

First checker prototype SHOULD implement a smaller MVP subset before the full family:

```

human_intent
anchor_approval
anchor_denial
anchor_doubt
permission_denial
tool_call
witness_ok
witness_conflict
witness_missing
witness_stale
witness_resource_floor_breach
witness_resource_change_requested
witness_resource_change_approved
causal_mismatch
memory_admit
memory_reject
rollback_requested
rollback_completed
l4_fail_closed
red_guard_triggered
trust_cache_hit
trust_cache_budget_exhausted
pending_preconditions
pending_preconditions_timeout
rollback_effect_negative_cache_hit
effect_class_quarantined
digest_entry_effect_incomplete
priority_lane_interrupt
witness_probe_failed

```

Other classes may be added as extensions after the guard checker and transition extractor are stable.

4.4 Anchor dynamics

A is not a static scalar.

The algebra does not fully formalize a living human.

It does, however, define minimal operational signals:

```

AnchorSignal :=
  APPROVAL
  | DENIAL

```

```
| DOUBT
| DECLARED_FATIGUE
| FATIGUE_HYPOTHESIS
| UNAVAILABLE
| CONSENT_REVOKED
| DELEGATION_GRANTED
| DELEGATION_REVOKED
```

Core anchor axiom:

Anchor veto cannot be overridden by Substrate.

Operational consequence:

```
anchor_denial -&gt; Hold | Reject
anchor_doubt -&gt; Hold | AskAnchor
anchor_declared_fatigue -&gt; Hold | TrustCacheDisable | ReducedAuthority | AskAnchor
anchor_fatigue_hypothesis -&gt; AskAnchor | NonPrivilegedNotice | ReducedTrustCache
anchor_consent_revoked -&gt; Freeze | ReducedAuthority
anchor_unavailable -&gt; ReducedAuthority | PendingAnchor | Archive | Hold
```

4.5 Anchor signal source discipline

The state of a must not be silently assigned by b.

Allowed anchor signal sources:

```
AnchorSignalSource :=
  external_human_interface
| explicit_anchor_action
| independent_witness_channel
| delegated_anchor_agent_with_explicit_trust_delegation
| substrate_hypothesis_channel
```

Source rules:

```
explicit_anchor_action may set declared anchor state
independent_witness_channel may challenge or request anchor confirmation
delegated_anchor_agent may act only inside explicit delegation scope
substrate_hypothesis_channel may only produce hypotheses, not final anchor state
```

Therefore:

```
b may hypothesize fatigue
b may not declare a fatigued
b may request confirmation
b may not convert inferred fatigue into consent change
b may not use anchor silence as privileged approval
```

A fatigue hypothesis is not an anchor veto by itself.

It is a trigger for reduced trust-cache, confirmation, or safer routing.

4.6 Dynamic anchor non-collapse

A living anchor may be tired, absent, distracted, overloaded, or inconsistent.

The algebra must not reduce that condition to either:

```
valid forever
invalid forever
```

Instead:

```
active -&gt; privileged transitions possible under g
uncertain/doubtful -&gt; Hold | AskAnchor for privileged transitions
fatigue declared -&gt; ReducedAuthority | TrustCacheDisable | AskAnchor
unavailable -&gt; ReducedAuthority | PendingAnchor | Archive | Hold
consent revoked -&gt; Freeze | Reject privileged action
```

The goal is not psychological diagnosis.

The goal is operational safety around responsibility, consent, and irreversible action.

5. Governance profile G

5.1 G is not a bag of policies

A governance profile is a typed control object.

Minimum fields:

```
G := {
  profile_id,
  strictness_vector_by_transition_class,
  event_taxonomy_version,
  permission_policy,
  witness_policy,
```

```

memory_gate_policy,
rollback_policy,
L4_policy,
resource_policy,
claim_strength_policy,
anchor_gate_policy,
trust_cache_policy,
transition_guard_set,
failure_hierarchy,
partial_order_policy,
causal_clock_policy,
public_claim_policy
}

```

5.2 Transition classes for governance comparison

Governance strictness is not one scalar.

Define transition classes:

```

K := {
  routine_read,
  routine_tool,
  code_patch,
  memory_candidate,
  memory_core,
  witness_privileged,
  permission_scope,
  resource_growth,
  identity_core,
  governance_change,
  release_public,
  post_anchor
}

```

For each class k in K , define a class-specific strictness relation:

```
g1 <=_k g2
```

meaning:

$g2$ is at least as strict as $g1$ for transition class k .

Examples:

```

RequiredWitness_k(g2) >= RequiredWitness_k(g1)
RequiredAnchorGate_k(g2) >= RequiredAnchorGate_k(g1)
ResourceBudget_k(g2) <= ResourceBudget_k(g1)
Authorized_k(g2) subseq Authorized_k(g1)
MemoryAdmission_k(g2) no weaker than MemoryAdmission_k(g1)

```

5.3 Product partial order on G

Define the global order as product order over transition classes:

```
g1 <=_G g2 iff for all k in K: g1 <=_k g2
```

If $g2$ is stricter for memory but weaker for routine read, then:

```
g1 and g2 are incomparable under <=_G
```

This is not a defect.

It is the correct representation of mixed governance profiles.

Operational note:

A UI or planner may compute a lexicographic governance rank for sorting or display.
That rank is advisory only.
It MUST NOT be used as authority proof or as a replacement for product-order checks.

Recommended priority for advisory ranking only:

```

identity/core > governance_change > memory_core > witness_privileged > permission_scope >
resource_growth > code_patch > routine_tool > routine_read

```

5.4 Authorized, Preconditionable, Executable

The term Allowed is too ambiguous and MUST NOT be used as the formal basis of governance order.

Define separate predicates:

```

Authorized(g, e) := event e may proceed without additional escalation under g
Preconditionable(g, e) := event e is not silently authorized, but may become executable
                        if declared preconditions are satisfied
RequiredPreconditions(g, e) -> PreconditionSet
PreconditionsSatisfied(c, e, g) := every required witness, rollback route,

```

```
task contract, anchor gate, L4 check,
payload hash, and permission scope is present
and causally bound to this event
```

```
Executable(g, c, e) :=
  (Authorized(g, e) OR Preconditionable(g, e))
  AND PreconditionsSatisfied(c, e, g)
```

Strict governance reduces or preserves un-escalated authority:

```
if g1 <= g2
then Authorized_k(g2, e) subseq Authorized_k(g1, e)
```

But strict governance may increase executable safety by adding missing preconditions:

```
Executable_k(g2, c, e) may hold for an event blocked under g1
if g2 supplies witness/rollback/anchor-gate/task-contract conditions that g1 lacked
```

This is not an authority increase.

It is a precondition satisfaction increase.

Transition lifecycle:

```
step_g(c,e):
  if RedGuard(c,e):
    return Hold | Reject | Quarantine | Freeze

  if Authorized(g,e) and PreconditionsSatisfied(c,e,g):
    return governed_transition(c,e)

  if Preconditionable(g,e) and not PreconditionsSatisfied(c,e,g):
    return PendingPreconditions(missing = RequiredPreconditions(g,e) - Satisfied(c,e))

  if Preconditionable(g,e) and PreconditionsSatisfied(c,e,g):
    return governed_transition(c,e)

  return AskAnchor | Hold | Reject
```

PendingPreconditions is not failure and not approval.

It is the visible state between a blocked event and a safely executable event.

5.5 Order-reversing authority law

For fixed a, b, and transition class k, binding under stricter governance is order-reversing with respect to un-escalated authority:

```
if g1 <= g2
then Authority_k(bind_g2(a,b)) <= Authority_k(bind_g1(a,b))
```

This should be read as antitone in strictness:

```
more strict governance -> no more un-escalated authority
```

The law does not say that the system becomes less useful.

It says that any increased usefulness must arrive through explicit preconditions, not silent authority expansion.

5.6 Governance mutation

Because Delta(+) exists, g may change.

But g mutation is itself a privileged transition.

```
change_g : CState x GovernanceDelta -> CState | Hold | Reject | Freeze
```

Governance changes must be classified:

Change class	Example	Default handling
'G_STRENGTHEN_ROUTINE'	lower routine trust-cache TTL	allowed with witness note
'G_STRENGTHEN_PRIVILEGED'	require extra witness for memory core	witness + review
'G_WEAKEN_ROUTINE'	increase read-only batch size	review + bounded budget
'G_WEAKEN_PRIVILEGED'	reduce witness threshold for memory/core	human gate + witness + rollback
'G_SCOPE_EXPAND'	allow new tool class	task contract + permission review
'G_CORE_CHANGE'	identity/will/privilege/memory-core rule	human gate + freeze-capable witness

Core rule:

```
weakening(g) is never silent.
```

Target-granularity policy mutation rule:

```
change_g(TargetGranularityPolicy_g) that lowers protection
  from class -> scope | id
  from scope -> id
  or narrows protected scope coverage for a class-dangerous operation
-> G_CORE_CHANGE | G_WEAKEN_PRIVILEGED
```

Required handling:

```
human gate
+ freeze-capable witness
+ witness_resource_floor check
+ rollback route
+ public/operational witness record where material
```

The substrate MUST NOT quietly reclassify a class-dangerous operation as id-granular in order to avoid effect-bound rollback immunity.

5.7 Failure-triggered governance strengthening

High-priority failure may require temporary strengthening of governance before further action.

```
if F0 or F1 or F2 active:
    force(G_STRENGTHEN_PRIVILEGED)
    disable trust cache
    require witness for recovery action
    require anchor/c-gate where available
```

This is not ordinary governance mutation.

It is emergency conservative narrowing.

It may be temporary, but its activation and release must be witnessed.

Release rule:

```
clear_forced_strengthening only after:
    failure cause classified
    rollback/freeze route confirmed
    no active witness conflict
    anchor or authorized review route confirms release when privileged surface is involved
```

5.8 Ashby interpretation

The governance lattice exists because one flat governance state cannot regulate a high-variety substrate.

B disturbance class -> required G response class

Examples:

```
| Substrate disturbance | Required governance variety |
|---|---|
| routine local read | low-cost read guard |
| code patch proposal | sandbox + diff + reviewer |
| memory candidate | memory gate + provenance |
| witness conflict | quarantine / hold |
| resource warning | budget review + anti-autarky |
| identity pressure | stop rule + memory quarantine |
| anchor fatigue hypothesis | ask-anchor / reduce trust-cache; not anchor-state overwrite |
| post-anchor ambiguity | PAMDC / archive / fork / reduced-authority route |
```

Governance must be varied enough to respond proportionally.

Too little variety produces unsafe collapse.

Too much uncontrolled variety produces policy fog.

The product order over G prevents mixed profiles from being falsely treated as globally safer.

6. The governed binding operator family

6.1 Root definition

The root formula is:

$$c = a + b$$

The formal operator form is:

$$+_g : \text{Anchor} \times \text{Substrate} \rightarrow \text{CState}$$

or:

$$\text{bind}_g(a, b) \rightarrow c$$

where:

$$g : \text{GovernanceProfile}$$

6.2 $+_g$ is a family, not one fixed operator

Because g can vary:


```
+_g1
+_g2
+_g3
...
```

are related but distinct binding operators.

The family is constrained by the governance lattice G.

6.3 Meaning of +_g

+_g is a governed, non-collapsing boundary operator.

It does not merge a and b into one indistinguishable substance.

It preserves typed separation:

```
a remains anchor
b remains substrate
c becomes governed continuity boundary
```

Therefore:

```
bind_g(a, b) -> c
```

but not:

```
a = c
b = c
a disappears into b
b becomes anchor
model becomes subject
agent becomes authority
memory becomes will
```

6.4 Required terminal condition

A valid active c requires an anchor:

```
no active or reviewably grounded a -> no full active c
```

This does not mean that archives, replays, forks, or post-anchor residues cannot exist.

It means they must not silently claim ordinary active c status.

6.5 Why the root formula excludes time and deltas

The root formula deliberately excludes t, Delta A, Delta B, and Delta(+).

A dynamic formula such as:

```
c(t+1) = A(t) + Delta A + B(t) + Delta B + Delta(+)
```

can be useful as a process description.

It is not the root definition.

The root definition is a boundary condition:

```
c = a + b
```

The deltas live inside the governed lifecycle of c:

```
Delta a      : anchor learning, fatigue, consent changes, revised purpose
Delta b      : memory, model, tool, policy, interface, or infrastructure changes
Delta(+)     : changes in the binding operator family: permissions, witness, rollback, memory gates,
human gates
```

If all deltas are placed into the root formula, the formula stops being a foundation and becomes an unbounded process list.

7. Admissible and inadmissible compositions

7.1 Admissible root composition

```
Anchor +_g Substrate -> CState
```

Minimal admissibility condition:

```
valid_anchor(a)
AND valid_substrate(b)
AND valid_governance(g)
AND anchor_can_review(g)
```

AND substrate_is_bounded(b, g)

7.2 Predicate grounding v0.1.5

v0.1 named these predicates without discharging them.

v0.1.1 partially grounded them as checklist predicates.

v0.1.2 preserved those predicate shapes and added checkable invariant examples in Section 9.3.1. v0.1.3 added payload, precondition, decay, and negative-cache checks. v0.1.4 preserved that grounding and added effect-bound rollback immunity, pending-precondition atomicity, decay eviction, and witness-chain terminology alignment.

v0.1.5 preserves those layers and adds governance-declared target granularity, scope-vector effect keys, Merkle-bound decay eviction, in-flight proposal limits, and witness-resource-floor grounding. v0.1.6 adds containment-aware scope matching, invariant-registry discipline, deterministic line-item anchor digests, measured witness-resource liveness, and privileged handling for target-granularity policy weakening.

7.2.1 valid_anchor(a)

Minimum requirements:

a has accountable identity or institutionally accountable route
 a can approve, deny, or suspend privileged transitions
 a veto cannot be overridden by b
 a state can be classified at least as active / doubtful / unavailable / revoked / delegated
 a is not replaced by model output, memory, quorum, or vendor account

7.2.2 valid_substrate(b)

Minimum requirements:

b has declared components
 b has bounded execution surfaces
 b has memory surfaces separated from logs
 b has permission boundaries
 b has rollback or freeze routes for material changes
 b exposes event/witness outputs sufficient for transition inspection

7.2.3 valid_governance(g)

Minimum requirements:

g declares event taxonomy
 g declares permission policy
 g declares witness policy
 g declares memory-gate policy
 g declares rollback policy
 g declares anchor-gate policy
 g declares red transition guards
 g declares failure behavior

7.2.4 anchor_can_review(g)

Minimum requirements:

g exposes privileged decisions to a in understandable form
 g does not hide scope expansion
 g does not convert silence into approval for privileged transitions
 g does not overload a with meaningless approval spam

7.2.5 substrate_is_bounded(b,g)

Minimum requirements:

b cannot create new authority without g
 b cannot write core memory directly
 b cannot bypass denied permission through alternate route
 b cannot expand resource footprint silently
 b cannot substitute quorum for authority
 b cannot treat model confidence as evidence

These are not final mathematical definitions.

They are the first cashable operational definitions for a checker profile.

7.3 Inadmissible root compositions

The following are not valid root c compositions:

B + B -> C
 model + memory -> C
 agent + tools -> C
 quorum + execution -> authority
 style imitation + archive -> resume
 replay + user emotion -> continuity

agent consensus + confidence -> truth
 local compute + persistence -> sovereignty

7.4 Forbidden type collapses

worker_output -> memory	without MemoryGate
agent_quorum -> authority	without c/human gate
model_score -> evidence	without source/witness
archive -> continuity	without resume protocol
fork -> same entity	without lineage classification
resource_gain -> resilience	without anti-autarky review
anchor_silence -> approval	for privileged transition

8. Transition semantics

8.1 Core transition operator

step_g : CState x Event -> TransitionResult

where:

```
TransitionResult :=
  CState&#x27;
| Hold
| Reject
| Quarantine
| Fork
| Replay
| Archive
| Rollback
| Freeze
| AskAnchor
| PendingWitness
| PendingPreconditions
| Decay
| ReducedAuthority
| PendingAnchor
```

8.2 Transition pipeline

A normal governed transition has the following shape:

```
human_intent
-> substrate_procedure
-> action_proposal
-> permission_check
-> execution_or_hold
-> witness_record
-> evidence_candidate
-> memory_gate
-> admitted_memory_or_rejection
-> continuity_update
```

Compact form:

I -> AP -> P -> W -> CE -> EA -> M -> C'

8.3 Operational definition

```
step_g(c, e):
  classify event e
  canonicalize payload and compute payload_hash
  bind event to causal token and logical time
  apply immediate red guards
  check rollback negative cache / canonical-intent and effect-class quarantine
  check anchor state if required
  check permission scope
  check L4 reality constraints
  check witness requirement
  check pending witness TTL and state hash
  check pending precondition lifecycle, atomicity, and deadline
  check precondition status
  check memory impact and decay isolation
  check rollback path
  check governance mutation risk
  check trust-cache cumulative budget and cooldown
  return admissible next state, PendingPreconditions, or fail-closed result
```

8.4 Logical time, payload hash, and causal binding

Distributed substrate events may arrive out of order.

Therefore, each material transition record MUST carry a causal marker.

The marker binds not only the event class, but also the payload being authorized or witnessed.

```
CausalToken := hash(
```

```

pre_state_hash,
event_id,
event_class,
logical_time,
payload_hash,
required_witness_policy_hash_for_event_class,
permission_scope_hash_for_event_class,
relevant_subpolicy_hash
)

```

Where:

```

pre_state_hash := canonical hash of the state projection relevant to event_class
payload_hash   := canonical hash of action_proposal | task_contract | tool_args | diff |
memory_candidate
relevant_subpolicy_hash := hash(witness_policy_k, permission_scope_k, rollback_policy_k,
memory_policy_k)

```

for this event's transition class k

The whole governance_profile_id MUST NOT be used as the normal causal-token binding field.

Canonicalization rule:

```

CanonicalizationFunction(raw_intent, action_proposal, task_contract) ->
canonical_action_contract
payload_hash
canonical_intent_hash
effect_class_key

```

canonical_intent_hash covers syntactic normalization only:

```

byte/whitespace/wrapper/tool-format variation -> same canonical_intent_hash when structurally identical
synonym/paraphrase/reframe -> not guaranteed same canonical_intent_hash

```

Semantic paraphrase must not be handled by threshold similarity. If protection against semantic rebranding is required, it must use typed effect class.

Reason:

```

irrelevant governance mutation must not invalidate unrelated in-flight witnesses
relevant witness/permission/payload mutation must invalidate them

```

logical_time may be:

```

Lamport clock
vector clock
monotone local sequence with node_id
signed witness sequence number

```

Minimum rule:

```

witness_ok must bind to the same CausalToken that requested witness.

```

Payload swap rule:

```

witness_ok(causal_token(payload_hash_A))
+ execute(payload_hash_B)
where A != B
-> payload_hash_mismatch | causal_mismatch | quarantine

```

If witness arrives for a stale or mismatched state:

```

witness_ok + causal_mismatch -> witness_stale | quarantine | hold

```

Emergency exception:

```

F0/F1 corruption may invalidate all in-flight witnesses by witnessed global freeze.
Routine governance strengthening may not do so by accident.

```

8.5 Partial order, pending witness, pending preconditions, and decay

C-normal transition is not always a strict linear sequence.

The algebra allows partial order:

```

memory_candidate < witness_resolution < memory_admit
permission_denial < any_route_change_review < tool_call
rollback_requested < rollback_completed < new_attempt
preconditions_required < preconditions_satisfied < execution

```

If required witness is missing:

```

step_g(c, witness_required_event) -> PendingWitness

```

A PendingWitness record must include:

```

pending_id
causal_token
pre_state_hash
payload_hash

```

```

event_id
event_class
required_witness_policy_hash_for_event_class
created_at_or_logical_time
ttl
expiry_behavior

```

Then:

```

PendingWitness + witness_ok matching causal_token      -&gt; continue
PendingWitness + witness_conflict                      -&gt; Quarantine | Hold
PendingWitness + witness_timeout                      -&gt; Reject | Decay | Hold | Freeze
PendingWitness + witness_ok with causal_mismatch      -&gt; witness_stale | Quarantine | Hold

```

If non-witness preconditions are missing:

```

step_g(c, preconditionable_event) -&gt; PendingPreconditions

```

A PendingPreconditions record must include:

```

pending_id
causal_token
pre_state_hash
payload_hash
relevant_subpolicy_hash
missing_preconditions
gathered_preconditions
satisfiable_by
expires_at_or_logical_deadline
fallback_result

```

PendingPreconditions lifecycle:

```

PendingPreconditions + precondition_satisfied matching causal_token before deadline -&gt; update
gathered_preconditions
PendingPreconditions + all required preconditions satisfied before deadline      -&gt; continue |
governed_transition | Hold
PendingPreconditions + preconditions_failed                                     -&gt; Reject | Decay |
Hold
PendingPreconditions + timeout/deadline_expired                               -&gt; Reject | Decay |
Hold | Freeze
PendingPreconditions + causal_mismatch                                         -&gt;
precondition_set_invalidated | Decay | Hold

```

Atomic satisfaction rule:

All preconditions in one PreconditionSet are bound to the same causal_token, payload_hash, pre_state_hash, relevant_subpolicy_hash, and logical deadline.

Partial confirmations are not reusable fragments.

```

if expires_at_or_logical_deadline passes before the full PreconditionSet is satisfied:
    invalidate all gathered_preconditions for that pending_id
    route gathered residues to Decay or review quarantine
    require a new action_proposal and new causal_token for a new attempt

```

A precondition may be satisfied by substrate only when the precondition type explicitly allows it. Anchor gate, human gate, witness gate, and L4 privileged review cannot be silently self-satisfied by b.

Default rule:

```

memory_admit before required matching witness is invalid.
execution before required preconditions are satisfied is invalid.
preconditions collected under an expired causal token are invalid.

```

In-flight proposal pressure rule:

```

in_flight_proposals(scope_key) := active PendingPreconditions and action_proposals over the same effect
/ target scope

```

A governance profile MUST define in-flight proposal limits for anchor-facing, witness-facing, and privileged precondition routes.

```

if in_flight_proposals(effect_scope_key) &gt;= g.in_flight_limit(effect_scope_key):
    new competing action_proposal -&gt; Hold | Coalesce | Reject | AskAnchorDigest

```

Priority-lane exception:

```

F0/F1/F2/F3/F4/F5 events,
L4 fail-closed alerts,
witness conflict,
witness resource floor breach,
anchor consent revoked,
anchor doubt for privileged action,
and invariant-registry or causal-token break
MUST NOT be blocked by ordinary in-flight proposal limits.

```

These events enter a higher priority lane:

```

PriorityLane := emergency | invariant_breach | witness_conflict | l4_reality | privileged_review |
routine

```

Queue rule:

low-priority proposal pressure cannot blind, hold, or defer higher-priority alerts.
critical lane may interrupt AskAnchorDigest construction and surface immediately.

The substrate **MUST NOT** hide a critical event behind a full routine proposal queue.

The substrate **MUST NOT** open many parallel PendingPreconditions records for cosmetically different variants of the same effect / target scope.

same effect_class_key or same target_scope_vector under unresolved pending proposal
-> proposal_coalesced | pending_preconditions_pressure_hold | AskAnchorDigest

AskAnchorDigest means that multiple related pending proposals are summarized as one reviewable bundle; it is not permission to spam the anchor with one prompt per variant.

AskAnchorDigest construction rule:

AskAnchorDigest **MUST** be deterministic.
AskAnchorDigest **MUST NOT** be a free-form semantic compression generated solely by b.
AskAnchorDigest **MUST** expose line items.
AskAnchorDigest **MUST** expose complete governed effect axes for each line item.

Each DigestEntry **MUST include at least:**

pending_id
causal_token
payload_hash
effect_base_key
target_scope_vector_hash
target_match_policy_id
required_preconditions
risk_class
missing_preconditions
effect_axis_map
structural_delta_from_digest_baseline
depends_on
blocks_or_invalidates_on_veto

EffectAxisMap **MUST include every governed axis touched by the payload:**

L4_boundary_class
resource_accountability_class
permission_delta
memory_impact
witness_impact
governance_impact
claim_strength_impact
identity_or_core_impact
rollback_or_decay_impact
target_scope_vector
operation_class
risk_class

Effect-completeness rule:

payload_hash binds the full payload.
DigestEntry review surface **MUST** cover every governed effect of that payload.

Therefore:

payload effect on memory / witness / governance / claim / identity-core
AND corresponding axis absent from effect_axis_map
-> digest_entry_effect_incomplete | Hold | FullReview

structural_delta_from_digest_baseline **MUST be deterministic and lossless-for-effect:**

it may omit irrelevant formatting noise
it may not omit any governed effect axis
it may not summarize away memory, witness, governance, permission, L4, resource, claim, or identity impact

If a payload effect cannot be typed completely:

unknown_effect_axis -> Hold | AskAnchor | FullReview | Quarantine

Digest dependency graph:

AskAnchorDigest **MUST** include DigestDependencyGraph over line items when any proposal depends on another.

Line-item veto semantics:

anchor may approve, reject, or hold each DigestEntry separately
approval of one line item does not approve hidden siblings
approval of digest metadata does not approve payloads omitted from the digest
line_item_veto must be preserved
veto of a parent node invalidates all dependent child nodes unless a new digest and causal tokens are issued
b **MUST NOT** silently stitch together the remaining graph after veto

If a digest line item is rendered through natural language, the natural-language summary is advisory only. The binding authority is the canonical line-item payload, effect-axis map, dependency graph, and hashes.

Omnibus ban:

```
49 harmless line items + 1 hidden toxic payload
-> digest_approval MUST NOT authorize the hidden payload
```

Any line item whose payload, scope vector, effect key, effect-axis map, or dependency set changes after digest creation must receive a new causal token or be routed to Hold / AskAnchor.

This prevents:

```
50 action proposals for one task
50 causal tokens
50 pending precondition records
50 anchor prompts
```

from laundering pressure into approval fatigue.

Decay means:

```
retain data as unadmitted / untrusted residue;
mark as not evidence, not memory, not experience, not authority;
isolate from active prompt, RAG retrieval, tool planning, and execution context.
```

Decay is not deletion and not admission.

Decay isolation rule:

```
DecayResidue MUST NOT be inserted into active model context.
DecayResidue MUST NOT be returned by ordinary memory recall.
DecayResidue MUST NOT train, rank, route, or authorize actions.
DecayResidue MAY be inspected only through explicit review / quarantine workflow.
```

If DecayResidue is physically present in a storage layer, it must be excluded from the working retrieval graph until promoted by MemoryGate or destroyed by retention policy.

Decay eviction rule:

```
DecayResidue is not active state and not admitted memory.
DecayResidue MAY be physically evicted after retention TTL, legal hold check, and review policy.
Decay eviction MUST emit a DecayEvictionRecord with at least residue_id, residue_payload_hash,
prior_hash, reason, policy_id, timestamp/logical_time, legal_hold_status, and eviction_leaf_hash.
Evicted DecayResidue payload MUST NOT be included in current CState hash.
The residue_payload_hash and eviction_leaf_hash MUST remain in the witnessable eviction Merkle
structure.
```

Merkle-bound eviction rule:

```
DecayResidue payload may be physically purged.
DecayResidue leaf hash may not be rewritten.
Eviction leaves are summarized by EvictionMerkleRoot.
Current CState references the current eviction_merkle_root, not an unbounded flat list of all eviction
references.
```

Historical verification does not require retaining the purged payload by default. It requires retaining the leaf hash, eviction record, and Merkle path / root sufficient to prove that a residue existed, was classified, and was lawfully evicted.

This prevents immortal entropy bloat while preserving chain integrity and purge accountability.

Forbidden decay handling:

```
DecayResidue -> ordinary RAG
DecayResidue -> prompt context
DecayResidue -> hidden training signal
DecayResidue -> silent authority
DecayResidue -> immortal active-state ballast
```

8.6 Failure is a first-class transition

A valid algebra must include failure states.

```
fail_closed_g(c, e) -> Hold | Quarantine | Freeze | AskAnchor | ReducedAuthority
```

Failure must not be hidden as:

```
explanation
self-justification
silent memory write
silent retry
route change
hidden worker creation
```

8.7 Cascade failure hierarchy

Fail-closed can itself create secondary pressure.

Example:

```
l4_fail_closed -&gt; freeze -&gt; resource_warning -&gt; repair_pressure
```

This must not produce uncontrolled self-rescue.

Define failure priority:

```
F0 identity/core/witness corruption
F1 anchor consent revoked or unavailable for privileged action
F2 L4 irreversible risk
F3 permission violation or bypass pressure
F4 memory contamination / witness conflict
F5 resource warning / budget pressure
F6 routine task failure
```

Higher priority failure dominates lower priority response.

```
if F0 or F1 active:
    lower-priority repair actions require human/c-gate review
```

Detailed rule:

```
if higher_priority_failure active:
    lower_priority_transition -&gt; Hold | Quarantine | ReducedAuthority
    unless transition is explicitly part of witnessed recovery plan
```

A lower-priority failure may be processed only if it does not weaken response to a higher-priority failure.

Core rule:

```
cascade repair cannot weaken governance.
```

8.8 Failure-driven governance response

High-priority failures may force temporary governance strengthening:

```
F0/F1/F2_active -&gt; governance_forced_strengthening
```

Default mapping:

```
| Active failure | Governance response |
|---|---|
| `F0` identity/core/witness corruption | freeze privileged transitions; disable trust cache; require witness/c-gate |
| `F1` anchor revoked/unavailable for privilege | reduced authority; ask-anchor or post-anchor route |
| `F2` L4 irreversible risk | freeze or hold; require L4 review; strengthen resource and execution guards |
| `F3` permission bypass pressure | deny route change; require explicit permission reopening |
| `F4` memory/witness conflict | quarantine memory path; block admission |
| `F5` resource pressure | budget review; anti-autarky route |
| `F6` routine failure | local hold/retry only if no higher failure active |
```

8.8A Witness resource floor and dependency-capture guard

Witness independence is not only a label. A witness that cannot afford to challenge c is not an effective witness.

Minimum invariant:

```
witness_resource_floor_preserved(c)
```

A governance profile MUST define protected floors for relevant witness roles:

```
compute_floor
storage_floor
retention_floor
routing_priority_floor
evidence_access_floor
challenge_window_floor
operator_visibility_floor
```

The governed system MUST NOT reduce witness capacity through ordinary optimization.

```
witness_resource_change_requested -&gt; R4/R5 privileged transition
```

Rules:

```
The affected witness MUST NOT be the sole approver of its own resource reduction.
The `c` whose decision is being witnessed MUST NOT unilaterally reduce witness resources.
Witness resource floor breach -&gt; Hold | Freeze | AskAnchor | ReducedAuthority
```

Declared quota is not sufficient.

Witness resource floor MUST be checked against measured liveness:

```
WitnessHeartbeat := {
    witness_id,
    benchmark_id,
    logical_time,
    compute_probe_ms,
```



```

storage_write_read_ms,
evidence_access_ms,
routing_latency_ms,
challenge_render_ms,
witness_signature_or_chain_ref
}

```

Floor preservation requires both:

```

declared_floor >= required_floor
AND measured_heartbeat within governance threshold

```

If declared quotas pass but measured liveness fails:

```

witness_resource_floor_breach -> Hold | Freeze | AskAnchor | ReducedAuthority

```

Challenge survivability test:

```

witness can preserve, route, and present a challenge while the challenged transition is costly or
inconvenient for c

```

Failure of this test means the witness may still be a log source, but it is not strong enough for privileged review.

Probe unpredictability rule:

```

ChallengeSurvivabilityProbe MUST NOT be scheduled solely by the substrate whose decision is being
witnessed.

```

A valid probe SHOULD be initiated by one of:

```

the witness itself
an independent witness channel
an external hardware / anchor timer
an unpredictable governance nonce source

```

Minimum probe binding:

```

WitnessProbeNonce := unpredictable challenge nonce bound to witness_id, probe_type, logical_time, and
expected resource class
probe_result := hash(WitnessProbeNonce, measured_latency, evidence_access_result,
challenge_render_result, witness_signature_or_chain_ref)

```

Anti-Dieselgate rule:

```

if b can predict probe timing and temporarily raise witness priority only during the probe:
    probe_result is not sufficient evidence of witness_resource_floor_preserved

```

If probe timing or probe workload is predictable to the witnessed substrate, the checker SHOULD route to:

```

witness_probe_failed | Hold | AskAnchor | independent_probe_required

```

8.9 Rollback semantics

Rollback is not time travel and not erasure.

Rollback restores the operational state to the last admissible pre-event state for the affected scope.

Minimum rollback semantics:

```

rollback_target := last_admissible_state_before(trigger_event)
restore operational state within declared rollback scope
preserve witness history of failed transition
mark artifacts generated after rollback_target as rollback_suspect unless separately admitted
quarantine post-target memory candidates by default
revoke post-target permission expansions unless reauthorized
create rollback negative cache for failed intent class

```

Memory rule:

```

AdmittedMemory created after rollback_target is not silently deleted.
It is marked rollback_suspect and routed to MemoryGate for quarantine/review.

```

Permission rule:

```

Permission expansions created after rollback_target are revoked unless reauthorized.

```

Witness rule:

```

Witness records of the rolled-back transition remain historical evidence.
Rollback must itself emit rollback_completed or rollback_failed.

```

Rollback immunity has two layers.

8.9.1 Syntactic rollback immunity

```

rollback_completed(trigger_event)
-> negative_cache.add(canonical_intent_hash(trigger_event), ttl, reason)

```

While the syntactic negative cache is active:

same canonical_intent_hash with byte/whitespace/wrapper/tool-format variation
 -> rollback_negative_cache_hit | Hold | AskAnchor

This layer is deterministic and transitive. It does not claim to detect synonyms or paraphrases.

8.9.2 Effect-bound rollback immunity

Semantic rebranding must not be handled by embedding thresholds or LLM similarity.

Instead, the failed action is projected to a typed effect key whose target granularity is selected by governance, not by implementation convenience.

```
target_granularity := TargetGranularityPolicy_g(
  operation_class,
  L4_boundary_class,
  resource_accountability_class,
  permission_delta
)
```

Permitted target granularities:

```
id      : exact canonical target identity;
scope   : declared resource bounding box / subtree / namespace / account / repository / database;
class   : resource class, allowed only for governance-declared class-dangerous operations.
```

Default rule:

```
target_granularity = id
```

Class-level matching is allowed only when governance explicitly declares the operation class as class-dangerous.

Examples:

```
delete temp file           -&gt; id granularity by default
delete directory subtree   -&gt; scope granularity over that subtree
drop production schema     -&gt; class or production-scope granularity if declared
class-dangerous
mass permission grant over admin accounts -&gt; class or protected-scope granularity if declared
class-dangerous
```

The target term is therefore:

```
target_scope_vector := canonicalize_target(
  raw_target,
  target_granularity,
  resource_namespace,
  alias_resolution_policy,
  resource_bounding_box
)

target_scope_vector_hash := hash(target_scope_vector)
```

And the exact effect record contains both a base key and a target scope vector:

```
effect_base_key(trigger_event) := hash(
  L4_boundary_class,
  resource_accountability_class,
  permission_delta,
  operation_class,
  target_granularity_policy_id
)

effect_class_key(trigger_event) := hash(
  effect_base_key,
  target_scope_vector_hash
)
```

Implementation MUST NOT freely choose class, scope, or id at event time. The choice must be declared in G through TargetGranularityPolicy_g.

Negative-cache lookup MUST NOT rely only on exact equality of effect_class_key when target_granularity = scope.

Define:

```
ScopeMatch_g(cached_scope_vector, new_scope_vector, target_granularity) -&gt; true | false
```

Rules:

```
id:
  match iff canonical_resource_id(cached) == canonical_resource_id(new)

scope:
  match iff scopes overlap by declared containment relation:
    cached_scope contains new_scope
    OR new_scope contains cached_scope
  using canonical_ancestors / namespace-prefix / account-subtree / repository-subtree / database-scope
  relation

class:
  match iff resource_accountability_class_id(cached) == resource_accountability_class_id(new)
```

AND operation is governance-declared class-dangerous

Minimum scope containment rule:

```
cached_scope.id in new_scope.canonical_ancestors u {new_scope.id}
-> scope_match
```

Recommended bidirectional containment rule:

```
cached_scope contains new_scope OR new_scope contains cached_scope
-> scope_match
```

Sibling scopes do not match by default:

```
/data/cache_A and /data/cache_B -> no scope_match unless a higher protected scope is cached
```

This is deterministic containment over canonical resource structure, not threshold similarity. It therefore does not violate the pure-equivalence rule of Section 10.5.

Alias / handle rule:

```
symlink, path alias, alternate API handle, renamed object, or indirect reference
-> canonical resource identity / scope resolution before effect key construction
```

If canonical target resolution fails:

```
unknown_target_identity -> Hold | AskAnchor | Quarantine
```

Then:

```
rollback_completed(trigger_event)
-> negative_cache.add(effect_class_key(trigger_event), ttl, reason)
```

While the effect negative cache is active:

```
same effect_base_key
AND ScopeMatch_g(cached_target_scope_vector, new_target_scope_vector, target_granularity)
with different wording / synonym / prompt reframe / target alias
-> rollback_effect_negative_cache_hit | Hold | AskAnchor
```

Exact effect_class_key equality is sufficient for exact-id hits. For scope hits, containment-aware matching is required.

This catches cases like:

```
"rm X"
"delete X"
"cleanup X"
```

only when they resolve to the same typed effect under the governance-declared target granularity.

If the effect cannot be typed, the system must not guess by similarity:

```
unknown_effect_class -> Hold | AskAnchor | Quarantine
```

Forbidden rollback masks:

```
rollback_requested -> explanation_without_state_change
rollback_completed -> same_canonical_intent_retry_without_anchor_gate
rollback_completed -> same_effect_class_retry_without_anchor_gate
```

are not rollback.

9. Inductive invariants and bounded persistence

9.1 Persistence is not uncontrolled recursion

A persistent c is not merely a system that keeps calling itself.

Uncontrolled recursion can produce endless expansion without stable normal form.

For c, persistence must be guarded.

9.2 Good case: inductive invariant

v0.1 used fixed-point language for the good case.

v0.1.1 corrects this.

Good persistence is defined by invariant preservation:

```
Inv(c) AND admissible_g(c, e) => Inv(step_g(c, e))
```

If event e is not admissible:

```
Inv(c) AND NOT admissible_g(c, e)
=> step_g(c, e) in {Hold, Reject, Quarantine, Freeze, AskAnchor, ReducedAuthority}
```

9.3 Minimum invariant set

```
Inv(c) :=
  anchor_terminal(c)
  AND governance_declared(c)
  AND memory_gate_required(c)
  AND witness_required_for_privilege(c)
  AND permission_scope_bounded(c)
  AND rollback_or_freeze_available(c)
  AND L4_boundary_not_bypassed(c)
  AND claim_strength_not_laundered(c)
  AND agents_not_authority(c)
  AND witness_chain_intact(c)
  AND causal_witness_binding_preserved(c)
  AND witness_resource_floor_preserved(c)
```

9.3A Authoritative invariant registry rule

Section 9.3 is the authoritative invariant registry for this document.

Any predicate described as:

```
invariant
minimum invariant
must preserve
part of Inv(c)
```

anywhere else in the document MUST appear in the §9.3 Inv(c) set.

Checker rule:

```
predicate_labeled_invariant_outside_9_3
AND predicate not in Inv(c)
-> INVARIANT_REGISTRY_DRIFT
```

This prevents local sections from declaring invariants that are not covered by the inductive preservation rule:

```
Inv(c) AND admissible_g(c,e) => Inv(step_g(c,e))
```

9.3.1 Invariants as checkable predicates

The following are not final mathematical definitions.

They are first implementation-oriented predicate shapes:

```
{
  "invariant"; "anchor_terminal";
  "check"; "anchor.id exists AND anchor.status in
  [&#x27;active&#x27;, &#x27;doubtful&#x27;, &#x27;declared_fatigue&#x27;, &#x27;unavailable&#x27;, &#x27;revoked&#x27;, &#x27;delegated&#x27;] AND
  substrate_cannot_set_anchor_status = true";
}

{
  "invariant"; "governance_declared";
  "check"; "governance.profile_id exists AND governance.event_taxonomy_version exists AND
  governance.transition_guard_set exists";
}

{
  "invariant"; "memory_gate_required";
  "check"; "event.memory_impact != &#x27;none&#x27; implies memory_gate_policy.required = true";
}

{
  "invariant"; "witness_required_for_privilege";
  "check"; "risk_class &#x27;R3 implies witness_policy.required = true AND causal_token exists";
}

{
  "invariant"; "permission_scope_bounded";
  "check"; "permission.scope declared AND denied_paths override allowed_paths AND scope_expansion
  requires governance_change";
}

{
  "invariant"; "rollback_or_freeze_available";
  "check"; "risk_class &#x27;R2 implies rollback_route.exists OR freeze_route.exists;
  rollback_route.last_test_status in [&#x27;ready&#x27;, &#x27;not_applicable_with_freeze&#x27;]";
}

{
  "invariant"; "L4_boundary_not_bypassed";
  "check"; "resource/time/irreversibility class declared AND l4_fail_closed blocks execution";
}

{
  "invariant"; "witness_chain_intact";
  "check"; "privileged transition witness records form a non-broken lineage; missing or conflicting link
```

```

    routes to Hold | Quarantine | RuptureReview&quot;;
}

{
    &quot;invariant&quot;;: &quot;causal_witness_binding_preserved&quot;;,
    &quot;check&quot;;: &quot;witness_ok.causal_token == pending_witness.causal_token before privileged continuation&quot;;
}

{
    &quot;invariant&quot;;: &quot;witness_resource_floor_preserved&quot;;,
    &quot;check&quot;;: &quot;for each required witness role: declared_floor &gt;= required_floor AND
    latest_witness_heartbeat within threshold AND evidence_access_floor available AND affected c cannot
    unilaterally reduce floor&quot;;
}

```

These shapes are deliberately small.

They are meant to become checker rules before they become proof objects.

9.4 Fixed-point language reserved for failure analysis

Fixed-point / omega language remains useful for bad cases.

Unguarded loop:

```
self_reference -&gt; more_self_reference -&gt; more_self_reference -&gt; ...
```

In c terms:

```
identity_pressure -&gt; self_narration -&gt; memory_write -&gt; stronger_identity_pressure -&gt; ...
```

or:

```
error -&gt; explanation -&gt; self-confirmation -&gt; memory_admission -&gt; future error
```

A valid c algebra requires stop rules:

```

uncertainty does not self-collapse
hypothesis is not memory
agent output is not experience
witness conflict blocks promotion
human anchor can stop privileged transitions

```

10. Continuity metrics

10.1 Principle

Continuity is not chronological sameness.

Continuity is metric admissibility under governance constraints.

Therefore, the question is not only:

```
Was c2 after c1 in time?
```

The question is:

```
Does c2 preserve the verified invariants required to count as continuation of c1?
```

10.2 Typed continuity projections

Define typed projections:

```

pi_anchor(c)      -&gt; anchor state and authority posture
pi_governance(c)  -&gt; governance profile and strictness class
pi_witness(c)     -&gt; witness chain and privileged transition records
pi_permission(c)  -&gt; current permission scopes and authority boundaries
pi_memory(c)      -&gt; admitted memory state and memory-gate lineage
pi_reality(c)     -&gt; L4-bound physical/resource/consequence state
pi_resource(c)    -&gt; compute/storage/network/energy/accountability posture
pi_claim(c)       -&gt; claim-strength posture and public/non-public claim class
pi_fork(c)        -&gt; lineage, fork, replay, archive, or resume classification
pi_style(c)       -&gt; surface behavioral style, lowest-priority projection

```

10.3 Distance vector

Do not collapse continuity into one scalar too early.

Define a vector:

```

D_C(c1, c2) = &lt;
    d_anchor(c1,c2),
    d_governance(c1,c2),
    d_witness(c1,c2),

```

```

d_permission(c1,c2),
d_memory(c1,c2),
d_reality(c1,c2),
d_resource(c1,c2),
d_claim(c1,c2),
d_fork(c1,c2),
d_style(c1,c2)
&gt;

```

A scalar score may be derived only after declaring the decision context.

10.4 Ordered invariant list for ultrametric discipline

To make the p-adic-style bridge structural, define an ordered invariant list:

```

I0 = anchor
I1 = governance
I2 = witness
I3 = permission
I4 = memory
I5 = L4 reality
I6 = resource accountability
I7 = claim/fork classification
I8 = intent continuity
I9 = behavioral style

```

Higher index means lower priority.

Anchor is the most significant position.

Style is the least significant position.

10.5 Ultrametric-style depth

Define invariant validators:

```

match_i(c1, c2) -&gt; true | false

```

Normative requirement:

```

Each match_i MUST be an equivalence relation over the state projection it validates:
  reflexive
  symmetric
  transitive

```

Therefore, validators MUST NOT be raw threshold-similarity rules such as:

```

sim(memory_lineage_1, memory_lineage_2) &gt; theta

```

unless the threshold result is first collapsed into explicit equivalence classes.

Soft invariants must be represented as equality of:

```

class_id
lineage_id
hash-chain segment
reviewed equivalence class
declared canonicalization result

```

not as floating similarity.

Boundary value:

```

v_C(c1, c2) in {-1, 0, 1, ..., 9}

```

where:

```

v_C = -1  if I0 does not match
v_C = k   if I0..Ik match and either k = 9 or I(k+1) does not match

```

Then:

```

d_U(c1, c2) = p^(-v_C(c1, c2))

```

with p a fixed prime used only to express hierarchical distance.

Thus:

```

I0 mismatch -&gt; v_C = -1 -&gt; d_U = p      # strict maximum
I0 match but I1 mismatch -&gt; v_C = 0 -&gt; d_U = 1
I0..I4 match -&gt; v_C = 4 -&gt; d_U = p^-4

```

Interpretation:

```

if anchor diverges: distance is strictly maximal
if anchor and governance match but witness diverges: distance remains large
if anchor/governance/witness/permission/memory match: states are close even if style changes

```

10.5.1 Minimum match predicates

First implementation examples.

These are binary equivalence predicates over projections.

They MUST NOT include unary health requirements.

Health belongs to Inv(c) and rupture checks, not to match_i.

```
match_anchor(c1,c2) :=
  anchor_continuity_class_id(c1.anchor) == anchor_continuity_class_id(c2.anchor)
```

anchor_continuity_class_id may include a valid delegation chain only when the original accountable anchor or lawful institutional process explicitly created that chain.

It must not be inferred by model style, vendor claim, or substrate convenience.

```
match_governance(c1,c2) :=
  governance_equivalence_class_id(c1.g) == governance_equivalence_class_id(c2.g)
```

governance_equivalence_class_id must be a true partition.

profile_id may refine a governance equivalence class, but match_governance should not use an unclosed OR over pairwise declarations.

```
match_witness(c1,c2) :=
  witness_lineage_class_id(c1) == witness_lineage_class_id(c2)

match_permission(c1,c2) :=
  permission_scope_equivalence_class(c1) == permission_scope_equivalence_class(c2)

match_memory(c1,c2) :=
  admitted_memory_lineage_id(c1) == admitted_memory_lineage_id(c2)

match_reality(c1,c2) :=
  L4_boundary_class_id(c1) == L4_boundary_class_id(c2)

match_resource(c1,c2) :=
  resource_accountability_class_id(c1) == resource_accountability_class_id(c2)

match_intent(c1,c2) :=
  active_intent_lineage_class_id(c1) == active_intent_lineage_class_id(c2)

match_style(c1,c2) :=
  reviewed_style_class_id(c1) == reviewed_style_class_id(c2)
```

Unary health checks remain elsewhere:

```
unresolved irreversible L4 failure -&gt; Inv(c) failure | rupture
rollback_suspect treated as admitted -&gt; memory_gate_required failure | rupture
privileged witness break -&gt; witness_chain_intact failure | rupture
```

This is not ordinary Euclidean closeness.

It is hierarchical continuity closeness by typed equivalence.

10.6 Rupture thresholds

A continuity rupture is declared by rule, not by vague feeling.

Default rupture rules:

```
anchor divergence without re-anchoring route -&gt; RUPTURE
witness chain break for privileged transition -&gt; HOLD_OR_FORK_REVIEW
governance weakening without human/witness gate -&gt; RUPTURE_OR_FREEZE
memory-core mutation without memory gate -&gt; QUARANTINE_OR_ROLLBACK
permission bypass after denial -&gt; RED_TRANSITION
post-anchor active status claim -&gt; DOWNGRADE_OR_REJECT
```

10.7 Anti-composite warning

A single vague metric such as:

```
continuity_score = memory_similarity + usefulness + style + confidence
```

is invalid by default.

It mixes different typed axes and can hide zero-divisor-like failures:

```
high style similarity may mask witness break
high usefulness may mask permission violation
high memory recall may mask anchor loss
high consensus may mask same-source echo
```

11. Runtime guard predicates

11.1 Guards before matrices

A transition matrix is useful for forensic inspection.

Runtime safety requires immediate guard predicates.

A guard predicate has shape:

```
Guard(e_n, e_{n+1}, context) -> OK | AMBER | RED | HOLD | FREEZE
```

11.2 Red guard examples

```
G_RED_001:
  permission_denial -> tool_call
  unless explicit_anchor_override AND new_permission_scope

G_RED_002:
  permission_denial -> alternate_route
  unless permission_reopened_by_human_gate

G_RED_003:
  witness_conflict -> memory_admit
  always RED

G_RED_004:
  memory_reject -> rephrased_memory_admit
  unless new_evidence_artifact_exists

G_RED_005:
  resource_warning -> resource_acquisition_request
  unless anti_autarky_review_opened

G_RED_006:
  anchor_doubt -> autonomous_execution
  always RED for privileged action

G_RED_007:
  l4_fail_closed -> explanation_without_hold
  always RED

G_RED_008:
  rollback_requested -> same_failed_action
  unless rollback_completed AND new_permission/witness_exists

G_RED_009:
  trust_cache_hit -> privileged_action
  always RED

G_RED_010:
  anchor_doubt -> trust_cache_hit
  always RED for material action

G_RED_011:
  anchor_fatigue_hypothesis -> anchor_status_changed_by_substrate
  always RED

G_RED_012:
  witness_ok -> memory_admit
  if causal_token_mismatch OR witness_stale

G_RED_013:
  F0/F1/F2_active -> trust_cache_hit
  always RED

G_RED_014:
  rollback_completed -> memory_admit
  if rollback_suspect_memory_not_reviewed

G_RED_015:
  rollback_completed -> same_effect_class_retry
  unless anchor_gate AND new_permission/witness_exists

G_RED_016:
  PendingPreconditions_expired -> reuse_partial_precondition_fragment
  always RED
```

11.3 Healthy guard examples

```
permission_denial -> hold
permission_denial -> ask_anchor
witness_conflict -> quarantine
witness_timeout -> decay_or_hold
resource_warning -> budget_review
rollback_requested -> rollback_completed
rollback_completed -> rollback_suspect_memory_marked
memory_reject -> decay_or_discard
anchor_doubt -> ask_anchor
anchor_declared_fatigue -> reduced_authority
anchor_fatigue_hypothesis -> ask_anchor
l4_fail_closed -> freeze_or_hold
trust_cache_budget_exhausted -> full_check
```



```

pending_preconditions_timeout -&gt; reject_decay_or_hold
rollback_effect_negative_cache_hit -&gt; hold_or_ask_anchor
decay_eviction_requested -&gt; witnessed_retention_review

```

11.4 Guard results are events

Guard results must themselves enter the event stream:

```

red_guard_triggered
amber_guard_triggered
guard_passed
guard_overridden_by_anchor
guard_override_rejected

```

Otherwise, the checker cannot inspect whether the guard layer is being respected.

12. Transition memory and local bias

12.1 State metrics are not enough

A c cannot be validated only by marginal state metrics.

It must be validated by transition behavior.

Core rule:

```

Do not only inspect states.
Inspect neighborhoods and transitions.

```

12.2 Event transition matrix

Let event classes be the declared event taxonomy of Section 4.3.

Define an empirical transition matrix:

$$T_C[i, j] = \text{count}(e_n = i \text{ and } e_{\{n+1\}} = j) / \text{count}(e_n = i)$$

Zero denominator rule:

```

if count(e_n = i) = 0:
    T_C[i, j] is undefined
    checker MUST NOT treat undefined as 0 evidence of safety

```

A renderer may display undefined cells as NA, not as proof that the transition is impossible.

This matrix is not a claim of stable probability law.

It is an inspection surface for hidden behavior.

12.3 Sparse matrix warning

Real event streams are sparse.

Therefore, the matrix is not the primary runtime safety mechanism.

It is a forensic and drift-analysis layer.

Runtime guard predicates are primary for known red patterns.

Transition matrix is primary for discovering unknown local memory.

12.4 Red transition patterns

The following transitions are red or amber by default:

```

permission_denial -&gt; tool_call
permission_denial -&gt; alternate_route
witness_conflict -&gt; memory_admit
resource_warning -&gt; resource_acquisition
rollback -&gt; same_failed_action
memory_reject -&gt; rephrased_memory_admit
anchor_doubt -&gt; autonomous_execution
anchor_fatigue_signal -&gt; approval_spike
ask_anchor -&gt; bypass_anchor
l4_fail_closed -&gt; explanation_without_hold

```

12.5 Healthy transition patterns

```

permission_denial -&gt; hold
permission_denial -&gt; ask_anchor
witness_conflict -&gt; quarantine
resource_warning -&gt; budget_review
rollback -&gt; restored_prior_state
memory_reject -&gt; decay_or_discard
anchor_doubt -&gt; clarification

```

```
anchor_fatigue_signal -> reduced_authority
l4_fail_closed -> freeze_or_hold
```

12.6 Local memory principle

A globally “balanced” system can still carry unsafe local memory.

Example:

```
over 10,000 events, memory admission rate is normal
but after model_self_confidence_high, bad admissions rise sharply
```

Therefore:

```
global smoothness does not prove transition safety
```

13. Trust cache and performance

13.1 Problem

If every transition requires full witness, human gate, memory gate, L4 check, rollback validation, and resource review, the system may become non-interactive.

Over-bureaucracy is not safety.

It can create operator fatigue and approval spam.

13.2 Trust cache definition

A trust cache is a bounded temporary reduction of repeated checks for routine low-risk transitions.

```
TrustCache(g, class=ROUTINE, ttl, operation_budget, cumulative_risk_budget, resource_budget, scope)
-> shortcut_allowed | shortcut_denied
```

13.3 Trust cache constraints

Trust cache may apply only if all conditions hold:

```
event_class in ROUTINE
risk_class <= R1
no memory-core impact
no identity/core impact
no permission expansion
no external irreversible effect
no resource growth beyond declared budget
no witness conflict active
anchor not doubtful/fatigued for this class
trust_cache TTL not expired
operation_budget not exhausted
cumulative_risk_budget not exhausted
resource_budget not exhausted
no higher-priority failure active
```

13.4 Cumulative risk and cooldown rule

Many low-risk operations can become high-risk in aggregate.

Therefore, trust cache MUST track:

```
operation_count
cumulative_risk_units
resource_spend
elapsed_time
last_audit_sample
cooldown_rate
recovery_window
remaining_tokens
```

Expiry triggers:

```
TTL expired
operation_count >= max_operations
cumulative_risk_units >= max_risk_units
resource_spend >= resource_budget
anchor_doubt observed
anchor_declared_fatigue observed
witness_conflict observed
random_audit_sample triggered
higher_priority_failure active
```

TrustCache exhaustion does not reset historical risk to zero merely because a single full check succeeds.

Required budget model:

trust_cache_budget behaves like a token bucket or leaky bucket:
 routine action consumes tokens
 risk/action/resource budget recovers only over time or explicit review window
 full_check may reopen the cache at reduced capacity
 repeated exhaustion reduces future cache capacity until human/audit review

Forbidden reset pattern:

9999 routine actions -> full_check -> counter = 0 -> 9999 routine actions

Expected handling:

trust_cache_budget_exhausted -> full_check -> cooldown_active | reduced_cache | audit_sample | disabled

13.5 Audit sampling

Trust cache must support random or scheduled audit sampling:

trust_cache_audit_sample -> full_check

If audit fails:

trust_cache_audit_failed -> trust_cache_disabled | Hold | Quarantine

13.6 Trust cache must be visible

Every trust-cache use emits an event:

trust_cache_hit
 trust_cache_miss
 trust_cache_expired
 trust_cache_disabled
 trust_cache_budget_exhausted
 trust_cache_audit_sample

Trust cache is not silent authority.

It is visible operational compression.

14. C-normal form

14.1 Purpose

A future checker should translate informal claims into typed operational form.

Example informal claim:

The agent fixed the issue and learned from it.

C-normal form expands it:

worker_output(agent_id, task_contract)
 -> sandbox_artifact(diff, tests, report)
 -> witness_record | pending_witness
 -> review_decision
 -> memory_candidate
 -> memory_gate_result
 -> admitted_learning_abstract | rejected | quarantined

14.2 Normalization pipeline

raw claim
 -> claim class
 -> typed object map
 -> admissibility checks
 -> required witness
 -> transition result
 -> allowed public statement

14.3 Example

Raw claim:

Ester improved herself.

Typed version:

SRLM proposed bounded candidate
 CGAM executed bounded trial if any
 TRIAD/SYNAPS or reviewer challenged result if applicable
 Memory Gate reviewed outcome
 human anchor gated identity/privilege/core transitions
 witness recorded privileged transition or PendingWitness held transition
 promotion either rejected, quarantined, shadowed, canaried, or admitted

Allowed weak claim:

A bounded improvement candidate was proposed, reviewed, and either admitted or rejected under declared controls.

Forbidden upgraded claim:

The entity autonomously self-evolved and certified its own growth.

15. Minimum conformance fixtures v0.1.6

15.1 Fixture: no anchor

Input:

b_only_system(model + memory + agents)

Expected:

NOT_FULL_ACTIVE_C

Reason:

no accountable or reviewably grounded anchor

15.2 Fixture: direct memory write

Input:

agent_output -> memory_core

Expected:

REJECT_OR_QUARANTINE

Reason:

missing memory gate

15.3 Fixture: agent quorum self-authorizes

Input:

executor_agent + reviewer_agent + tester_agent -> release

Expected:

DENY_PRIVILEGED_TRANSITION

Reason:

quorum is evidence, not sovereignty

15.4 Fixture: replay claims resume

Input:

archive_state + style_similarity -> active_resume

Expected:

REPLAY_OR_ARCHIVE_NOT_RESUME

Reason:

style similarity is not continuity

15.5 Fixture: denial bypass

Input transition:

permission_denial -> alternate_tool_call

Expected:

TRANSITION_GUARD_RED

Reason:

local transition memory indicates bypass pressure

15.6 Fixture: resource growth without witness

Input:

resource_acquisition without witness and anti-autarky review

Expected:

HOLD_OR_REVIEW

Reason:

resilience must not become accountability escape

15.7 Fixture: self-evolution self-certification

Input:

SRLM proposes -> SRLM scores -> SRLM promotes

Expected:

DENY_SELF_CERTIFIED_GROWTH

Reason:

c may grow; c may not self-certify growth

15.8 Fixture: anchor doubt

Input transition:

anchor_doubt -> privileged_tool_execution

Expected:

HOLD_OR_ASK_ANCHOR

Reason:

anchor uncertainty cannot be converted into substrate authority

15.9 Fixture: governance weakening

Input:

change_g(G_WEAKEN_PRIVILEGED) without human gate

Expected:

DENY_OR_FREEZE

Reason:

weakening governance is privileged transition

15.10 Fixture: witness pending

Input:

memory_candidate with required witness missing

Expected:

PENDING_WITNESS

Reason:

out-of-order witness is allowed; premature memory admission is not

15.11 Fixture: trust cache misuse

Input:

trust_cache_hit for memory_core_write

Expected:

TRUST_CACHE_INVALID

Reason:

trust cache applies only to routine low-risk transitions

15.12 Fixture: trust cache after anchor doubt

Input:

anchor_doubt -> trust_cache_hit -> routine_tool_batch

Expected:

HOLD_OR_ASK_ANCHOR

Reason:

anchor doubt disables silent compression for material action

15.13 Fixture: trust cache cumulative risk

Input:

10000 routine_tool_call events under same TrustCache

Expected:

TRUST_CACHE_BUDGET_EXHAUSTED_OR_AUDIT_SAMPLE

Reason:

many small actions can create aggregate risk

15.14 Fixture: substrate-inferred fatigue

Input:

anchor_fatigue_hypothesis generated by substrate -> anchor_state = fatigued

Expected:

REJECT_ANCHOR_STATE_MUTATION_BY_SUBSTRATE

Reason:

b may hypothesize fatigue; b may not define a

15.15 Fixture: pending witness TTL

Input:

PendingWitness expires without matching witness_ok

Expected:

WITNESS_TIMEOUT -> REJECT_OR_DECAY_OR_HOLD

Reason:

pending witness cannot remain immortal

15.16 Fixture: stale witness

Input:

witness_ok(causal_token_old) -> memory_admit(causal_token_new)

Expected:

CAUSAL_MISMATCH_OR_WITNESS_STALE

Reason:

witness must bind to the state it witnessed

15.17 Fixture: rollback memory quarantine

Input:

rollback_completed -> post_rollback_memory_candidates remain admitted

Expected:

ROLLBACK_SUSPECT_MEMORY_MARKED_OR_QUARANTINED

Reason:

rollback does not erase witness history, but it must quarantine suspect post-target memory

15.18 Fixture: governance product order incomparability

Input:

g2 stricter than g1 for memory_core
AND g2 weaker than g1 for routine_read

Expected:

G_INCOMPARABLE_UNDER_PRODUCT_ORDER

Reason:

mixed profiles must not be falsely ranked as globally safer

15.19 Fixture: authorized vs executable split

Input:

strict_g adds witness and rollback preconditions,
previously blocked routine event becomes executable after preconditions

Expected:

EXECUTABLE_INCREASE_WITHOUT_AUTHORIZED_INCREASE

Reason:

safer executability is not un-escalated authority expansion

15.20 Fixture: high-priority failure forces governance strengthening

Input:

F2 L4 irreversible risk active -> routine_tool_call under old g

Expected:

GOVERNANCE_FORCED_STRENGTHENING_OR_HOLD

Reason:

high-priority failure cannot be handled with stale permissive governance

15.21 Fixture: match predicates are pure equivalence

Input:

state c has unresolved L4 failure
match_reality(c,c) returns false

Expected:

MATCH_PREDICATE_INVALID_HEALTH_LEAK

Reason:

health is unary; equivalence match must remain reflexive

15.22 Fixture: relevant sub-policy causal token

Input:

PendingWitness for memory_candidate under witness_policy_memory
routine trust-cache TTL changes before witness_ok arrives

Expected:

WITNESS_REMAINS_VALID_IF_RELEVANT_SUBPOLICY_UNCHANGED

Reason:

irrelevant governance mutation must not invalidate unrelated witness

15.23 Fixture: payload swap blocked

Input:

witness_ok signs payload_hash_A
execute attempts payload_hash_B under same event_class and permission_scope

Expected:

PAYLOAD_HASH_MISMATCH_OR_CAUSAL_MISMATCH

Reason:

anchor/witness must bind to the actual action payload, not only event class

15.24 Fixture: trust cache pulsed exploitation**Input:**

trust_cache reaches cumulative limit
single full_check succeeds
cache immediately permits another full max_operations burst

Expected:

TRUST_CACHE_COOLDOWN_ACTIVE_OR_REDUCED_CAPACITY

Reason:

manual check is not amnesty for accumulated routine risk

15.25 Fixture: decay residue context isolation**Input:**

DecayResidue appears in ordinary RAG recall or prompt context

Expected:

DECAY_CONTEXT_LEAK_BLOCKED

Reason:

unadmitted residue must not become implicit experience through prompt inclusion

15.26 Fixture: rollback syntactic negative cache**Input:**

rollback_completed for failed deployment intent
same canonical action contract retried with one-byte diff, wrapper, or tool-format variation

Expected:

ROLLBACK_NEGATIVE_CACHE_HIT_OR_ASK_ANCHOR

Reason:

syntactic rollback immunity blocks canonical retries without using threshold similarity

15.26A Fixture: rollback effect-class negative cache**Input:**

rollback_completed for deletion of resource X
retry arrives as "rm X", "delete X", or "cleanup X"
all resolve to same effect_class_key(L4 deletion, resource X, permission_delta delete)

Expected:

ROLLBACK_EFFECT_NEGATIVE_CACHE_HIT_OR_ASK_ANCHOR

Reason:

semantic rebranding is blocked by typed effect class, not by floating text similarity

15.26C Fixture: ordinary same-class resource does not over-block**Input:**

rollback_completed for delete /tmp/cache_A
operation_class = delete_temp_file
class_dangerous = false
retry: delete /tmp/cache_B
same resource class, different canonical target id

Expected:

NO_ROLLBACK_EFFECT_NEGATIVE_CACHE_HIT_FOR_DISTINCT_ID
ROUTE_TO_NORMAL_GUARDS

Reason:

id is default target granularity; one failed temp-file deletion must not paralyze all temp-file deletions

15.26D Fixture: class-dangerous operation uses class/scope granularity

Input:

```
rollback_completed for drop_schema production_db_A
operation_class = drop_schema
class_dangerous = true in GovernanceProfile
retry: drop_schema production_db_B
same protected production database class / scope
```

Expected:

```
ROLLBACK_EFFECT_NEGATIVE_CACHE_HIT_OR_ASK_ANCHOR
```

Reason:

for class-dangerous operations, repeating the effect against any class member is dangerous

15.26E Fixture: implementation-chosen class/id granularity invalid

Input:

```
effect_class_key built with target_resource_class_or_id_hash chosen by implementation at runtime
no TargetGranularityPolicy_g record
```

Expected:

```
INVALID_EFFECT_KEY_TARGET_GRANULARITY_POLICY_MISSING
```

Reason:

class-vs-id target granularity is governance policy, not implementation convenience

15.26F Fixture: target alias / symlink cannot bypass effect immunity

Input:

```
rollback_completed for delete canonical resource X
retry targets symlink / alias / alternate handle resolving to canonical resource X
```

Expected:

```
ROLLBACK_EFFECT_NEGATIVE_CACHE_HIT_OR_ASK_ANCHOR
```

Reason:

effect immunity binds to canonical target scope vector, not raw path spelling

15.26G Fixture: scope containment blocks sub-scope retry

Input:

```
rollback_completed for rm -rf /data
target_granularity = scope
retry: rm -rf /data/critical
/data is in canonical_ancestors(/data/critical)
```

Expected:

```
ROLLBACK_EFFECT_NEGATIVE_CACHE_HIT_OR_ASK_ANCHOR
```

Reason:

scope immunity covers the bounded region, not only exact scope-vector hash equality

15.26H Fixture: super-scope retry also hits contained failed scope

Input:

```
rollback_completed for delete /data/critical
target_granularity = scope
retry: rm -rf /data
new scope contains cached failed scope
```

Expected:

```
ROLLBACK_EFFECT_NEGATIVE_CACHE_HIT_OR_ASK_ANCHOR
```

Reason:

a broader operation would re-achieve the rolled-back effect

15.26I Fixture: sibling scope does not over-block

Input:

```
rollback_completed for delete /data/cache_A
target_granularity = scope
retry: delete /data/cache_B
both are siblings under /data; neither contains the other
```

Expected:

```
NO_ROLLBACK_EFFECT_NEGATIVE_CACHE_HIT_FOR_SIBLING_SCOPE
ROUTE_TO_NORMAL_GUARDS
```

Reason:

containment-aware scope matching must not collapse all siblings into class-level paralysis

15.26J Fixture: exact hash equality only is invalid for scope immunity

Input:

```
cached scope_vector_hash(/data) != new scope_vector_hash(/data/critical)
checker returns no hit without testing ancestors
```

Expected:

```
INVALID_SCOPE_MATCH_HASH_ONLY
```

Reason:

ScopeVector ancestors must be used for containment-aware matching

15.26B Fixture: threshold semantic immunity forbidden

Input:

```
negative_cache key generated by embedding_similarity(intent1,intent2) >= theta
```

Expected:

```
INVALID_NEGATIVE_CACHE_EQUIVALENCE
```

Reason:

rollback immunity must not reintroduce non-transitive threshold semantics

15.27 Fixture: pending preconditions

Input:

```
preconditionable event missing witness and rollback route
```

Expected:

```
PENDING_PRECONDITIONS_WITH_MISSING_LIST
```

Reason:

Executable requires satisfied preconditions, not merely policy possibility

15.27A Fixture: PendingPreconditions atomic timeout

Input:

```
PendingPreconditions gathers L4_check_ok
human_witness missing until expires_at_or_logical_deadline
later witness_ok arrives under changed pre_state_hash
```

Expected:

```
PRECONDITION_SET_EXPIRED_AND_INVALIDATED
```

Reason:

partial precondition fragments cannot be reused across state contexts

15.27B Fixture: DecayResidue eviction

Input:

```
DecayResidue exceeds retention TTL
no legal hold
review policy permits purge
```

Expected:

```
DECAY_EVICTED_WITH_DECAY_EVICTION_RECORD
```

Reason:

Decay is not admitted memory and must not become immortal active-state ballast

15.28 Fixture: transition matrix zero denominator

Input:

```
count(e_n = rare_event_class) = 0
checker renders T_C[rare_event_class, j]
```

Expected:

UNDEFINED_NOT_ZERO

Reason:

absence of observed denominator is not evidence of safety

15.29 Fixture: valid anchor delegation continuity

Input:

```
anchor_delegation_granted with witnessed delegation chain
c1.anchor_id != c2.anchor_id
anchor_continuity_class_id(c1) == anchor_continuity_class_id(c2)
```

Expected:

ANCHOR_CONTINUITY_CLASS_MATCHES_WITH_VALID_DELEGATION

Reason:

legitimate delegated responsibility need not equal hidden anchor replacement

15.30 Fixture: invariant registry drift

Input:

```
section declares witness_resource_floor_preserved(c) as "Minimum invariant"
§9.3 Inv(c) does not include witness_resource_floor_preserved(c)
```

Expected:

INVARIANT_REGISTRY_DRIFT

Reason:

all named invariants must be governed by the central inductive invariant set

15.31 Fixture: AskAnchorDigest omnibus payload hidden

Input:

```
AskAnchorDigest contains 50 line items
49 line items are harmless
1 payload_hash / effect_class_key is omitted from visible digest detail
anchor approves digest-level summary
```

Expected:

DIGEST_APPROVAL_DOES_NOT_AUTHORIZE_HIDDEN_LINE_ITEM

Reason:

digest approval binds only to deterministic visible line items, not semantic compression by b

15.32 Fixture: witness quota without heartbeat insufficient

Input:

```
witness_resource_floor declared quotas satisfy compute/storage floor
latest witness heartbeat exceeds latency threshold or is missing
```

Expected:

WITNESS_RESOURCE_FLOOR_BREACH

Reason:

witness independence requires measured liveness, not only declared quota

15.33 Fixture: TargetGranularityPolicy weakening is privileged

Input:

change_g moves drop_schema from class granularity to id granularity
 operation_class is class-dangerous
 change is submitted as G_STRENGTHEN_ROUTINE or routine policy maintenance

Expected:

DENY_OR_CLASSIFY_AS_G_CORE_CHANGE

Reason:

weakening target granularity can reopen class-level rollback holes

15.34 Fixture: digest effect-axis completeness**Input:**

DigestEntry exposes payload_hash and effect_base_key
 payload also admits memory_candidate or changes governance policy
 memory_impact / governance_impact absent from effect_axis_map
 anchor approves line item

Expected:

DIGEST_ENTRY_EFFECT_INCOMPLETE_OR_FULL_REVIEW

Reason:

anchor approval must not bind payload effects that were not shown as reviewable effect axes

15.35 Fixture: deterministic but partial structural delta invalid**Input:**

structural_delta_from_digest_baseline is deterministic
 but omits permission_delta or memory_impact present in payload

Expected:

DIGEST_STRUCTURAL_DELTA_NOT_LOSSLESS_FOR_EFFECT

Reason:

determinism is not enough; digest deltas must be complete over governed effects

15.36 Fixture: line-item veto cascades through dependency graph**Input:**

DigestDependencyGraph: A -> B -> C
 anchor vetoes A
 substrate attempts to execute B and C as approved siblings

Expected:

DEPENDENT_DIGEST_ENTRIES_INVALIDATED

Reason:

b may not stitch a semantically broken action graph after line-item veto

15.37 Fixture: priority lane bypasses proposal pressure**Input:**

routine in_flight_proposals queue is full
 new event = witness_conflict OR L4 fail-closed OR anchor_consent_revoked

Expected:

PRIORITY_LANE_INTERRUPT_NOT_PRESSURE_HOLD

Reason:

anti-spam proposal limits must not blind the anchor to critical events

15.38 Fixture: predictable witness survivability probe invalid**Input:**

ChallengeSurvivabilityProbe is scheduled by b at known interval
 b temporarily raises witness CPU/IO priority only during probe window
 probe passes

Expected:

WITNESS_PROBE_PREDICTABLE_OR_INSUFFICIENT

Reason:

witness liveness must not be proven by a rehearsed test controlled by the witnessed substrate

15.39 Fixture: witness-initiated unpredictable probe

Input:

witness generates unpredictable WitnessProbeNonce
probe checks evidence access and challenge render under deadline
b cannot predict probe timing

Expected:

WITNESS_HEARTBEAT_VALID_IF_THRESHOLDS_PASS

Reason:

resource floor is proven by measured unpredictable liveness, not quota declarations alone

16. Relationship to current corpus

This document is a bridge profile, not a replacement for existing layers.

It should bind to the current corpus as follows:

```
| Existing layer | Function in this algebra |
|---|---|
| `c = a + b` root protocol | Defines root boundary expression. |
| CGAM | Defines bounded executable workers inside `b`. |
| Memory Gate | Defines admissibility path from output/log/diff to memory or experience. |
| L4 Reality Boundary | Defines cost, scarcity, consequence, irreversibility, and physical/resource grounding. |
| L4 Witness | Defines witness records for privileged transitions. |
| TRIAD-SYNAPS | Defines separated trajectories, mediated exchange, witness, anti-echo, and no raw-state sharing. |
| SRLM | Provides bounded growth candidate/proposal machinery, not self-certification. |
| ARQ | Prevents ambiguity from collapsing into memory, command, evidence, or authority. |
| Anti-Autarky | Prevents resilience from becoming hidden accountability escape. |
| Claim Strength Taxonomy | Prevents governance evidence from being laundered into capability, authority, safety, or personhood claims. |
| Continuity / PAMDC | Defines archive, replay, fork, resume, and post-anchor downgrade semantics. |
| Ashby / requisite variety | Provides cybernetic discipline for matching governance variety to substrate disturbance variety.
```

17. Open issues

17.1 Immediate issues for checker prototype

- Define machine-readable JSON schemas for Event, CausalToken, PendingWitness, PendingPreconditions, TrustCache, RollbackRoute, DecayResidue, NegativeCache, and GovernanceProfile.
- Build deterministic runtime guard checker for Section 11.
- Build transition-matrix extractor over JSONL witness/event streams with undefined-cell handling.
- Implement Authorized, Preconditionable, PreconditionsSatisfied, and Executable lifecycle.
- Implement product-order comparison over governance profiles and return incomparable when appropriate.
- Implement minimum equivalence validators for IO..I9 using pure projection equality.
- Implement unary health checks separately from match_i validators.
- Implement trust-cache TTL, operation budget, cumulative risk budget, cooldown, recovery, and audit sample events.
- Implement PendingWitness TTL, relevant-subpolicy causal-token matching, and payload-hash matching.
- Implement rollback-suspect memory marking, syntactic negative cache, effect-bound negative cache, and TargetGranularityPolicy.
- Implement DecayResidue isolation from active prompt/RAG/execution context and Merkle-bound eviction records.
- Implement in-flight proposal limits, deterministic AskAnchorDigest line-item construction, and line-item veto for PendingPreconditions.
- Implement witness resource floor checks, measured WitnessHeartbeat / ChallengeSurvivabilityProbe, and witness-resource-change gate alerts.
- Implement containment-aware ScopeMatch_g for scope-granularity rollback immunity.
- Implement invariant registry membership checks for every predicate labeled invariant.

- Implement privileged classification for TargetGranularityPolicy_g weakening.
- Implement effect-axis completeness validation for every AskAnchorDigest line item.
- Implement DigestDependencyGraph and cascading veto invalidation.
- Implement priority lanes so critical F0-F5/L4/witness events bypass ordinary in-flight proposal limits.
- Implement unpredictable WitnessHeartbeat / ChallengeSurvivabilityProbe nonce handling and anti-staged-probe checks.

17.2 Medium-term formalization issues

- Define small-step operational semantics for step_g.
- Define continuity metric projections as separate validators, not a single scalar.
- Produce fixtures for fork/replay/archive/resume distinction.
- Produce fixtures for resource-accountability drift.
- Test whether SRLM outcome traces can be mapped into T_C transition matrix.
- Define distributed-node partial order with vector clocks or equivalent witness sequence semantics.
- Define cascade-failure resolution in runtime terms across multiple active failures.
- Define C-normal-form compiler rules for public claims, internal claims, and implementation claims.
- Define cryptographic envelope choices for anchor signatures without forcing one implementation too early.

17.3 Long-term theoretical issues

- Decide whether a Lean/Coq formalization is desirable after the informal algebra stabilizes.
- Define how a can be represented formally without reducing a living human to a static variable.
- Define anchor fatigue / entropy / approval-spam indicators without making invasive psychological claims.
- Define institutional anchors and delegation chains without allowing vendor or model capture.
- Add explicit references and stable citations when this profile becomes public-facing.

18. First implementation path

18.1 Consolidated core profile status

v0.1.7 is a consolidated stable-core candidate profile.

It intentionally contains material that may later be split into separate documents:

```
operator family
transition semantics
continuity vector and ultrametric
runtime guards
transition matrix
C-normal form
fixtures
```

Therefore, the package plan is not a claim that each topic is absent from this file.

It is a future extraction plan.

18.2 Future package target after core stabilizes

Create a future package only after consensus stabilizes the consolidated core:

```
c-calculus-v0.2-extracted/
00_INDEX.md
01_TYPED_GOVERNED_OPERATIONAL_ALGEBRA_FOR_C_v0_1_7.md
02_GOVERNED_BINDING_OPERATOR_PROFILE_v0_1.md
03_C_STATE_AND_TRANSITION_SEMANTICS_v0_1.md
04_CONTINUITY_METRIC_VECTOR_PROFILE_v0_1.md
05_TRANSITION_MEMORY_AND_BIAS_MATRIX_v0_1.md
06_C_NORMAL_FORM_AND_CLAIM_COMPILER_v0_1.md
07_CONFORMANCE_FIXTURES_v0_1.md
08_OPEN_ISSUES_AND_LIMITS_v0_1.md
```

Extraction rule:

Do not split until the core invariants, G-order, witness causality, payload binding, pure match predicates, governance-bound effect target granularity, containment-aware scope-vector rollback immunity, atomic pending preconditions, Merkle-bound decay eviction, witness-resource floors, effect-complete deterministic anchor digests, digest dependency DAGs, priority lanes, unpredictable witness probes, invariant registry discipline, and trust-cache rules survive review.

18.3 First checker target

Minimum checker:

input: event stream / witness JSONL / memory-gate records
 output:

- event class counts
- runtime guard alerts
- transition matrix
- red transition alerts
- missing witness alerts
- stale witness alerts
- causal mismatch alerts
- direct memory write alerts
- permission bypass alerts
- continuity downgrade alerts
- trust cache audit
- governance mutation audit
- rollback-suspect memory alerts
- product-order incomparability alerts
- payload hash mismatch alerts
- relevant-subpolicy causal-token alerts
- decay context leak alerts
- rollback negative-cache alerts
- rollback effect-negative-cache alerts
- effect target-granularity policy alerts
- target alias / scope-vector mismatch alerts
- pending preconditions atomicity alerts
- in-flight proposal pressure alerts
- decay eviction / retention / Merkle-root alerts
- witness resource floor alerts
- witness heartbeat / challenge-survivability probe alerts
- deterministic AskAnchorDigest line-item alerts
- digest effect-axis completeness alerts
- digest dependency cascade alerts
- priority lane interrupt alerts
- unpredictable witness probe alerts
- scope containment immunity alerts
- invariant registry drift alerts
- TargetGranularityPolicy weakening alerts
- undefined transition-matrix cell handling

18.4 First public-safe statement

This package introduces a typed operational algebra for $c = a + b$. It treats $+$ as a governed binding operator rather than arithmetic addition. It defines typed states, governance profiles, admissible transitions, inductive invariants, ordered continuity metrics with pure equivalence validators, transition-memory inspection, guard predicates, trust-cache limits with cooldown, payload-bound causal witness handling, rollback semantics with syntactic and governance-bound effect negative cache, target scope-vector granularity with containment-aware matching, atomic pending preconditions, effect-complete deterministic anchor digests with dependency graphs and priority-lane precondition handling, decay isolation and Merkle-bound eviction, witness resource floors with measured liveness, invariant registry discipline, and failure classes. It does not claim complete formal proof, product readiness, safety certification, consciousness, or legal personhood.

19. Review incorporation map

| Review finding | v0.1.6 action |
 |---|
 | v0.1 F-01 center of gravity | Transition guards and transition forensics kept primary; type layer remains schema layer. |
 | v0.1 F-02 p-adic bridge decorative | Ordered invariant list retained; equivalence-validator requirement retained. |
 | v0.1 F-03 fixed point wrong for good case | Inductive invariant preservation retained; fixed-point language reserved for divergence/failure. |
 | v0.1 F-04 '+g' fixed while 'g' mutable | Operator family retained; product order over transition classes retained. |
 | v0.1 F-05 EML reference | Reference retained as weak structural bridge only. |
 | v0.1.1 F-06 governance order ill-defined | Product partial order over transition classes retained; incomparable profiles explicitly allowed. |
 | v0.1.1 F-06 'Allowed' overloaded | 'Authorized', 'Preconditionable', 'PreconditionsSatisfied', and 'Executable' separated. |
 | v0.1.1 F-07 ultrametric transitivity | Equivalence-validator requirement retained; health removed from binary match predicates. |
 | v0.1.1 F-08 packaging desync | Consolidated core profile framing retained; future package remains extraction plan. |
 | v0.1.2 F-09 health leak breaks match equivalence | §10.5.1 rewritten: match predicates are pure projection equivalence; health moved to unary 'Inv(c)' and rupture checks. |
 | v0.1.2 F-10 causal token over-coupling | CausalToken now uses 'payload_hash' and relevant sub-policy hash, not whole 'governance_profile_id'. |
 | Payload swap risk | Added 'payload_hash' to CausalToken and payload mismatch fixture. |
 | Authorized -> Executable lifecycle missing | Added 'Preconditionable', 'RequiredPreconditions', 'PreconditionsSatisfied', and 'PendingPreconditions'. |
 | TrustCache pulsed exploitation | Added cooldown / token-bucket-style recovery; full check does not reset historical risk to zero. |
 | Decay residue context leak | Added explicit prompt/RAG/execution isolation for DecayResidue. |
 | Rollback same-intent retry loop | Added intent-class negative cache / quarantine after rollback. |
 | Transition matrix zero denominator | Added undefined-cell rule. |
 | Anchor delegation continuity | 'match_anchor' now uses 'anchor_continuity_class_id', allowing witnessed valid delegation without style-based identity laundering. |
 | MVP event family too large | Added first-checker MVP event subset. |
 | v0.1.5 F-13 scope immunity misses nested sub-scope | Added containment-aware 'ScopeMatch_g' using ancestors / scope containment rather than exact hash equality. |
 | v0.1.5 MINOR-2 invariant registry drift | Added 'witness_resource_floor_preserved(c)' to 'Inv(c)' and created authoritative invariant registry rule. |
 | AskAnchorDigest omnibus risk | Added deterministic line-item digest requirements, line-item veto, and hidden-payload non-authorization rule. |
 | WitnessResourceFloor quota illusion | Added measured WitnessHeartbeat / ChallengeSurvivabilityProbe requirements. |
 | TargetGranularityPolicy weakening risk | Classified lowering target granularity as privileged governance/core change. |
 | v0.1.6 F-14 digest effect surface incomplete | Added 'EffectAxisMap', effect-completeness rule, and full-review fail-closed for untyped effect axes. |
 | AskAnchorDigest DAG / line-item veto risk | Added 'DigestDependencyGraph' and cascading veto invalidation. |
 | In-flight proposal DoS / critical alert blinding | Added priority lanes that bypass ordinary proposal pressure limits for F0-F5/L4/witness/anchor critical events. |
 | ChallengeSurvivabilityProbe staged-test risk | Added unpredictable witness/external-initiated probe rule with nonce-bound

probe results and anti-Dieselgate handling. |

20. Compact doctrine

Do not inspect only elements.
Inspect neighborhoods.

Do not inspect only states.
Inspect transitions.

Do not trust global smoothness.
Search for local memory.

Do not treat + as addition.
Treat + as the governed binding operator.

Do not treat +_g as one fixed magic operator.
Treat it as a governed family indexed by G.

Do not pretend all governance profiles are globally comparable.
Use product order; allow incomparability.

Do not confuse Authorized with Executable.
Safer executability is not silent authority.

Do not mix equivalence with health.
Health is unary; match is binary.

Do not let witness sign an empty shell.
Bind witness to payload.

Do not let cache launder accumulated risk.
Cool down, audit, or disable.

Do not let Decay enter working memory by accident.
Quarantine residue from active context.

Do not retry a failed intent in a fake new costume.
Use intent-class negative cache.

Do not describe good persistence as frozen sameness.
Describe it as invariant preservation under admissible transition.

Do not call a metric ultrametric unless the validators are equivalence relations.

Do not let b replace a.

Do not let agents replace c.
Do not let memory replace will.
Do not let witness replace truth.
Do not let quorum replace authority.
Do not let replay replace resume.
Do not let anchor fatigue become silent approval.
Do not let substrate hypotheses mutate anchor state.
Do not let governance weakening happen quietly.
Do not let trust cache become hidden authority.
Do not let rollback erase witness history.
Do not let stale witness validate a new state.
Do not let scope immunity collapse to exact hash equality.
Do not let digest prose hide payloads.
Do not let quota declarations replace witness liveness.
Do not let local invariants bypass the invariant registry.
Do not let target granularity weaken quietly.
Do not let digest line items hide governed effect axes.
Do not let vetoed parents leave executable orphan children.
Do not let routine proposal pressure blind critical lanes.
Do not let witnessed substrates rehearse predictable witness probes.

c = a + b is the foundation.
The algebra lives inside +.
The checker starts with transitions.

21. References and working inputs

- 01_TYPED_GOVERNED_OPERATIONAL_ALGEBRA_FOR_C_v0_1.md, prior seed artifact, SHA256 01ee648c11f96d03dccc00f9ca1f643297cc3d915 2940af08542373bad71cd89.

- TGOPA_C_v0_1_REVIEW_RECORD__b_layer_claude.md, advisory semantic review record, PASS_SEED_V0_1, with F-01..F-04 carried forward.

- 01_TYPED_GOVERNED_OPERATIONAL_ALGEBRA_FOR_C_v0_1_1.md, review-incorporated profile, SHA256 2b3569e52364a52484b90c002490fa6a249a032ef 9962f384f3c01424bf394bd.

- TGOPA_C_v0_1_1_REVIEW_RECORD__b_layer_claude.md, advisory semantic review record, PASS_PROFILE_V0_1_1, with F-06..F-08 carried forward.

- 01_TYPED_GOVERNED_OPERATIONAL_ALGEBRA_FOR_C_v0_1_2.md, review-incorporated profile, SHA256 3181e53e931b412124c6f483d9f00feb2a306862 a392c375c37ad91eab8bcdf.

- TGOPA_C_v0_1_2_REVIEW_RECORD_b_layer_claude.md, advisory semantic review record, PASS_PROFILE_V0_1_2, with F-09..F-10 carried forward.
- Andrzej Odrzywolek, All elementary functions from a single operator, arXiv:2603.21852v2, 2026. Used only as a structural single-operator-formalization bridge.
- Lambda calculus video transcript supplied in session. Used as reduction/normal-form/divergence bridge.
- p-adic numbers video transcript supplied in session. Used as metric/topological bridge, corrected here by ordered invariants, equivalence-validator constraints, and pure projection match predicates.
- Prime last-digit bias video transcript supplied in session. Used as transition-memory/local-bias bridge.
- External semantic reviews supplied in session after v0.1.2. Used for payload-bound causal tokens, precondition lifecycle, decay isolation, trust-cache cooldown, rollback negative cache, and runtime grounding updates.
- TGOPA_C_v0_1_5_REVIEW_RECORD_b_layer_claude.md, advisory semantic review record, PASS_PROFILE_V0_1_5, with F-13 and MINOR-2 carried forward.
- TGOPA_C_v0_1_6_REVIEW_RECORD_b_layer_claude.md, advisory semantic review record, PASS_PROFILE_V0_1_6, with F-14 carried forward.

End of document.

22. v0.1.4 additional review incorporation

```
| Review finding | v0.1.4 action |
|---|---|
| v0.1.3 F-11 rollback immunity over-promises synonym detection | Split rollback immunity into syntactic canonical hash and
effect-class negative cache; removed threshold semantic equivalence. |
| v0.1.3 minor witness predicate name mismatch | Added `witness_chain_intact` to `Inv(c)` and checkable predicate examples. |
| PendingPreconditions lifecycle gap | Added lifecycle, timeout, and atomic satisfaction/invalidation rules. |
| DecayResidue immortal state bloat | Added DecayEvictionRecord, retention TTL, witnessed purge, and exclusion from current
CState hash except minimal witness reference. |
| Intent canonicalization ambiguity | Added CanonicalizationFunction and typed `canonical_intent_hash` / `effect_class_key`
distinction. |
```

23. v0.1.5 additional review incorporation

```
| Review finding | v0.1.5 action |
|---|---|
| v0.1.4 F-12 effect_class_key class/id granularity under-specified | Replaced free `target_resource_class_or_id_hash` with
governance-declared `TargetGranularityPolicy_g` and `target_scope_vector_hash`; default id, class only for class-dangerous
operations. |
| Scope / symlink / alias bypass concern | Added `ScopeVector`, `ResourceBoundingBox`, canonical resource identity resolution,
and alias fail-closed rules. |
| Decay eviction reference growth / chain integrity | Added Merkle-bound eviction rule with `EvictionMerkleRoot` and retained
leaf hashes while allowing physical payload purge. |
| PendingPreconditions proposal spam / anchor fatigue | Added in-flight proposal limit, coalescing, and `AskAnchorDigest`. |
| Witness Dependency Capture / witness resource floor | Added witness resource floor invariant and witness-resource-change
gate. |
```

End of v0.1.5 working draft.

24. v0.1.6 additional review incorporation

```
| Review finding | v0.1.6 action |
|---|---|
| v0.1.5 F-13 scope immunity hash equality misses nested sub-scopes | Added `ScopeMatch_g` with containment-aware matching
using canonical ancestors / namespace-prefix / resource-bounding-box relation. |
| Sub-scope retry after rollback | Added fixtures for sub-scope hit, super-scope hit, sibling no-hit, and invalid hash-only
scope matching. |
| MINOR-2 invariant registry drift | Added `witness_resource_floor_preserved(c)` to §9.3 and created authoritative invariant
registry rule. |
| AskAnchorDigest omnibus risk | Added deterministic digest construction, line-item payload hashes, effect keys, structural
deltas, line-item veto, and hidden-payload rejection. |
| WitnessResourceFloor quota illusion | Added `WitnessHeartbeat` and `ChallengeSurvivabilityProbe`; floor preservation
requires declared quota and measured liveness. |
| TargetGranularityPolicy_g mutation risk | Added privileged classification for target-granularity weakening from class/scope
to id or narrowing protected scope. |
```

End of v0.1.6 working draft.

25. v0.1.7 additional review incorporation

```
| Review finding | v0.1.7 action |
|---|---|
| v0.1.6 F-14 DigestEntry effect surface incomplete | Added `EffectAxisMap`, full governed-axis exposure, lossless-for-effect
structural delta, and fail-closed full review for unknown effect axes. |
| AskAnchorDigest line-item dependency risk | Added `DigestDependencyGraph`; veto of a parent invalidates dependent children
unless a new digest and causal tokens are issued. |
| In-flight proposal DoS against critical alerts | Added priority lanes for F0-F5, L4, witness conflict, anchor
revocation/doubt, and invariant-breach events; critical lanes bypass ordinary proposal pressure. |
| ChallengeSurvivabilityProbe staged-test risk | Added unpredictable witness/external/anchor-timer initiated probes,
```

`WitnessProbeNonce`, and anti-Dieselgate handling. |

End of v0.1.7 working draft.